



Training parallelo e distribuito di Reti Neurali

Dai principi di Parallel Computing a Data Parallelism, Model Parallelism e Training su Cluster



Sicurezza Informatica e Tecnologie Cloud Casi di Studio nella PA e nell'Industria

 12 maggio
9:30 - 17:00

 **UNISA | Sede di Avellino**
Palazzo di Città
Piazza del Popolo

Interventi:
Agenzia per la Cybersicurezza Nazionale |
Amazon Web Services | Fortinet | IBM | Regione Campania |
Terna | VMEngine

 **Attestato di partecipazione**

Navetta da Fisciano per Avellino 

LAUREA MAGISTRALE

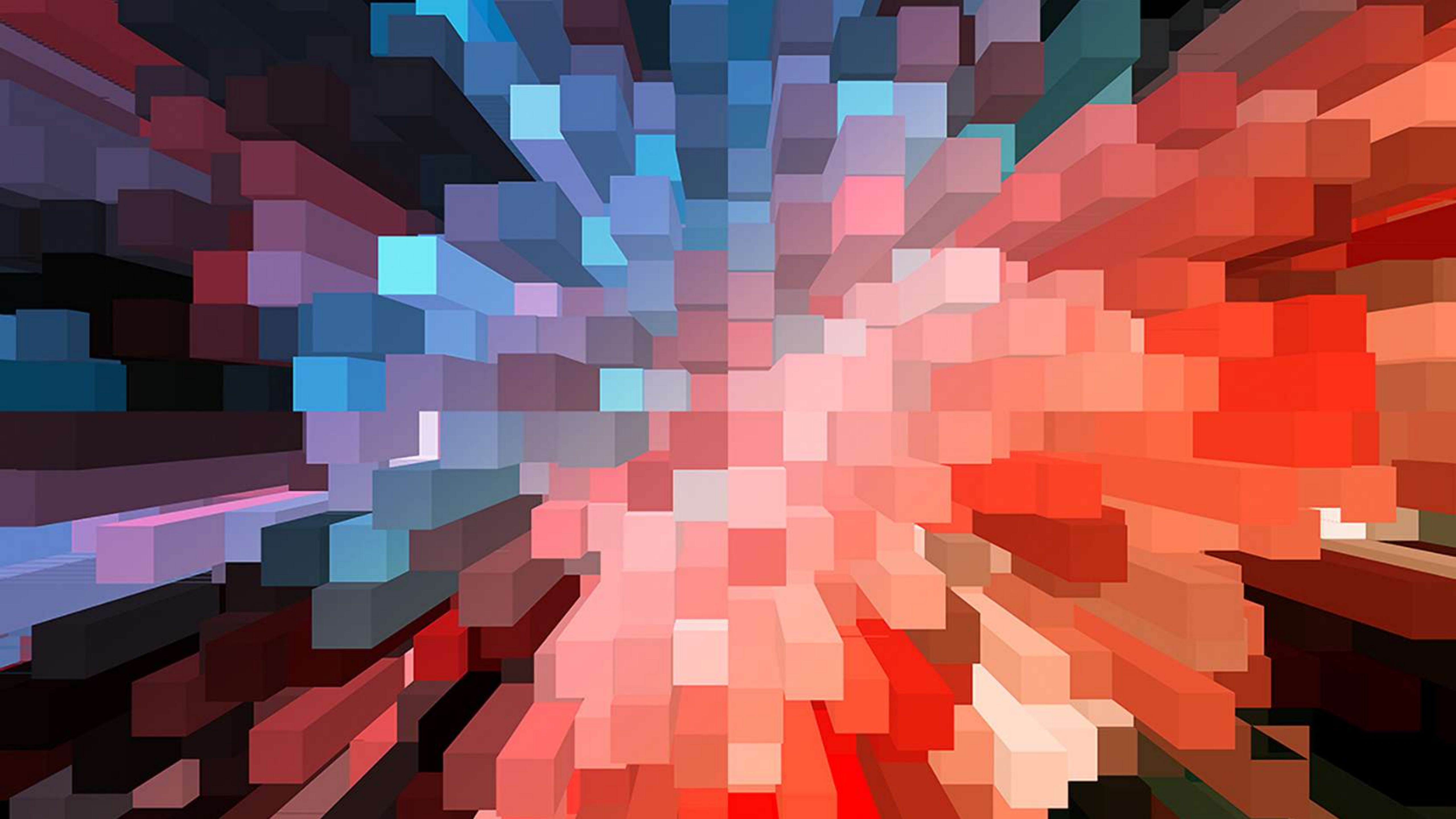
SICUREZZA INFORMATICA E TECNOLOGIE CLOUD (CLASSE LM-66)

DIPARTIMENTO DI INFORMATICA PER LA **TERZA MISSIONE**

Outline

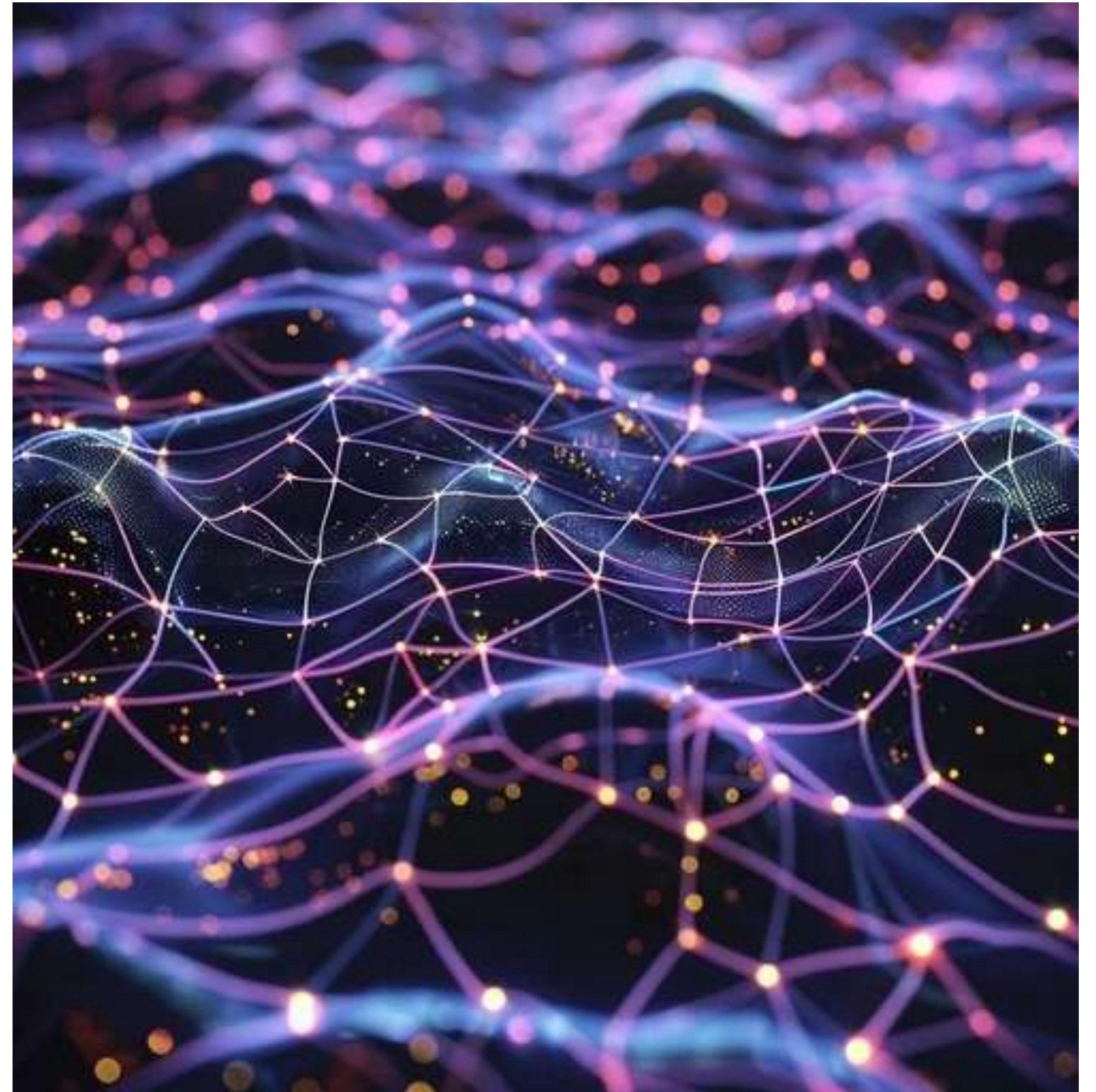
Training parallelo e distribuito di Reti Neurali

- Introduzione alle Reti Neurali
- Motivazione al parallelismo
- Ciclo di training di una rete neurale
- Tipologie di training Parallelo e Distribuito
 - Data Parallelism
 - Model Parallelism
 - Pipeline Parallelism
 - Tensor Parallelism



Introduzione alle reti neurali

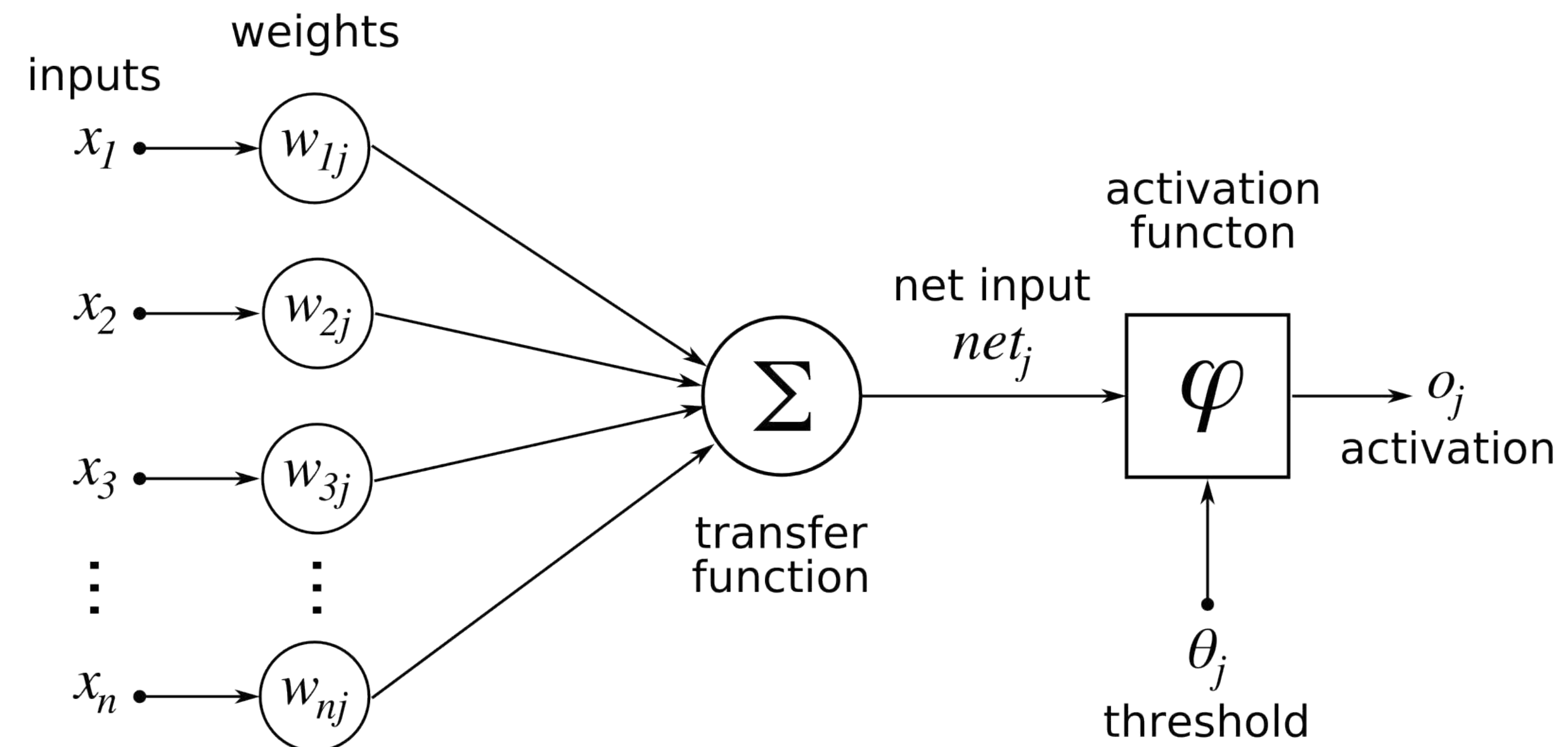
- Cos'è una rete neurale?
- Processo di addestramento di una rete neurale



Reti neurali

Cos'è una rete neurale?

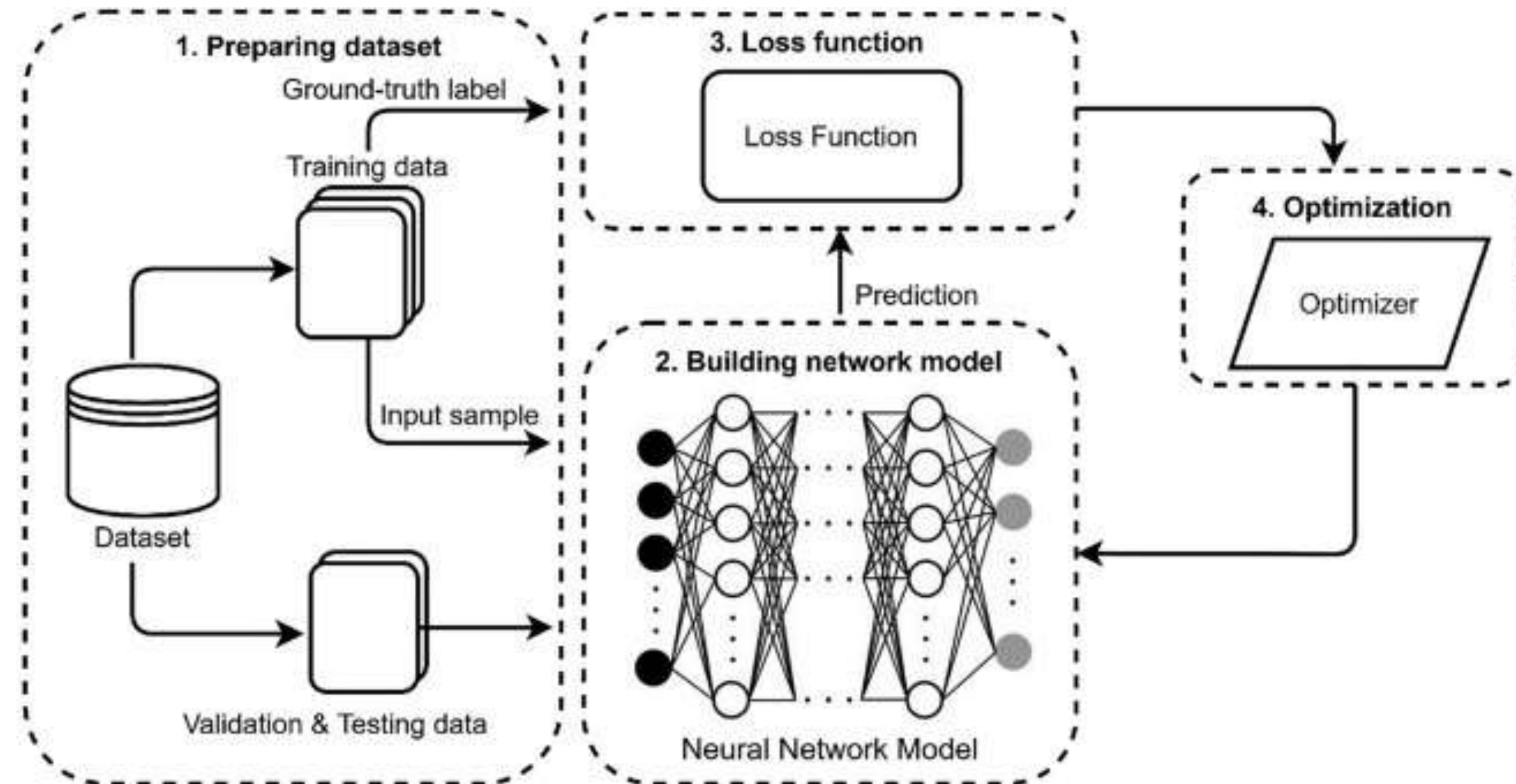
- Una rete neurale è un sistema che **approssima funzioni** complesse attraverso combinazioni di trasformazioni non lineari.
- Una rete neurale è un sistema di unità interconnesse che trasformano input in output attraverso **pesi modificabili**.
- Una rete neurale è un **algoritmo di apprendimento** automatico composto da neuroni artificiali organizzati in layer (livelli).
- Una rete neurale è una struttura matematica che **riconosce schemi e relazioni** nei dati tramite l'addestramento.
- Una rete neurale è un **modello che impara a svolgere un compito** regolando progressivamente i propri parametri interni.



Reti neurali

Cos'è una rete neurale?

- Le reti neurali sono solitamente organizzate in layer.
- Ogni layer processa l'input e lo passa al Layer successivo.
 - Le operazioni effettuate nei Layer sono regolate sulla base di **parametri** numerici w_i
- L'**addestramento** (o training) è il processo algoritmo in cui i parametri vengono modificati per risolvere un task al meglio



Reti neurali

Processo di Training

- Durante il training, vengono eseguite in maniera iterativa le seguenti operazioni:
 - Leggere un batch di dati (sottoinsiemi del dataset originale)
 - Eseguire una **forward pass**
 - Calcolare il valore della **loss function**
 - Eseguire una **backward pass**
 - **Calcolo dei Gradienti**
 - **Aggiornare i parametri**
- Durante l'addestramento, il dataset può essere letto più volte

Reti neurali

Processo di Training

- I cicli del processo di addestramento sono principalmente due:
- **STEP** -> Una semplice iterazione su un batch di dati
- **EPOCA** -> Una lettura completa del dataset

Un'epoca può essere formata da uno o più step di addestramento

Reti neurali

Processo di Training → Forward Pass

- Nel **forward pass**, il modello riceve un input, ad esempio un'immagine, una frase o un vettore di feature, e lo propaga attraverso i suoi layer.
- Ogni layer applica trasformazioni matematiche ai dati, **usando i pesi correnti** del modello.
- Il risultato finale è una previsione: una classe, un valore numerico, una probabilità o una sequenza di token.
- Il forward pass risponde alla domanda:
 - “Cosa predice il modello con i pesi attuali?”

Input $x \rightarrow$ Layer 1 (W_1) $\rightarrow$ Layer 2 (W_2) $\rightarrow \dots \rightarrow$ Layer n (W_n) $\rightarrow$ Predizione $\hat{y}$

Reti neurali

Processo di Training > Calcolo della Loss

- Dopo aver prodotto una previsione, il modello confronta il risultato ottenuto con il valore corretto, chiamato target o label.
- La funzione di loss quantifica quanto la previsione sia lontana dalla risposta desiderata. Un valore di loss alto indica che il modello sta sbagliando molto; un valore basso indica che la previsione è più vicina al target.
- La loss trasforma l'errore del modello in un numero da minimizzare.

$$\text{Predizione } \hat{y} + \text{Target } y \rightarrow \text{Loss } L(\hat{y}, y)$$

Reti neurali

Processo di Training > Backward pass

- Il backward pass calcola come l'errore finale dipende dai parametri interni del modello.
- A partire dalla loss, **l'errore viene propagato** all'indietro attraverso i layer, **dall'ultimo verso il primo**.
- Questo processo usa la regola della catena per determinare quanto ogni peso abbia contribuito all'errore complessivo.
- Se il forward pass produce la previsione, la loss stima l'errore, il backward pass spiega quali parti del modello sono responsabili dell'errore.

Valore della *loss function* → Layer N → ... → Layer 2 → Layer 1

Reti neurali

Processo di Training > Backward pass > Calcolo dei Gradienti

- I gradienti **indicano quanto** e in quale direzione **ciascun parametro del modello dovrebbe cambiare per ridurre la loss**.
- Per ogni parametro viene calcolata una **derivata parziale della loss rispetto a quel peso**.
- Un gradiente positivo o negativo indica la direzione della variazione; la sua grandezza indica quanto quel parametro influenza l'errore.

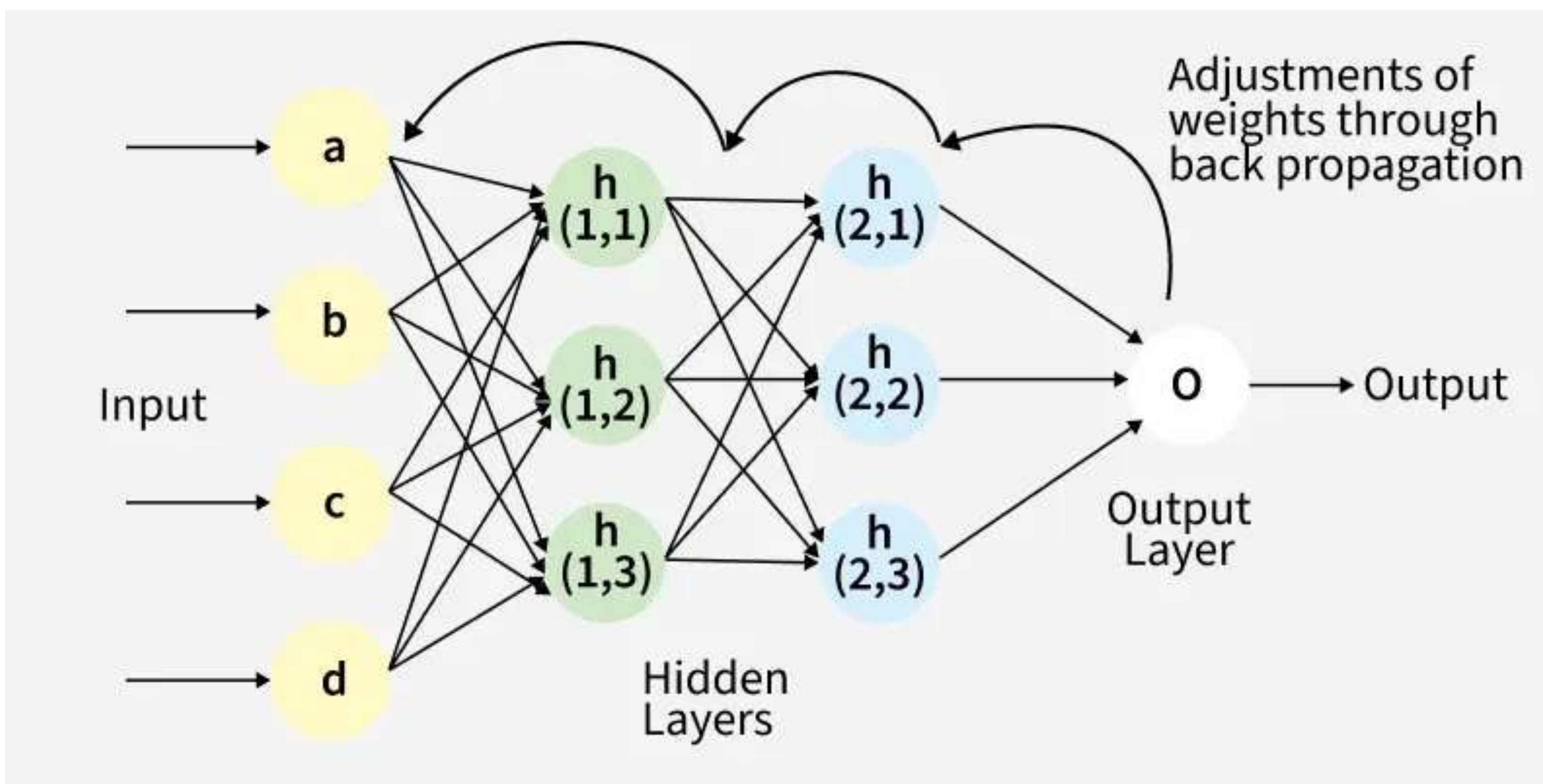
$$\begin{array}{l} \text{Peso } w_3 \rightarrow \text{gradiente } \frac{\partial L}{\partial w_3} \\ \text{Peso } w_2 \rightarrow \text{gradiente } \frac{\partial L}{\partial w_2} \\ \text{Peso } w_1 \rightarrow \text{gradiente } \frac{\partial L}{\partial w_1} \end{array}$$

gradiente = variazione della loss rispetto a un peso ($\partial L / \partial w$)

I gradienti sono il “segnale” che guida l'apprendimento del modello.

Reti neurali

Processo di Training > Overview



Reti neurali

Processo di Training > Aggiornamento dei pesi

- Dopo aver calcolato i gradienti
 - Tramite un **algoritmo di ottimizzazione**, vengono aggiornati i pesi del modello
 - L'obiettivo è modificare i parametri in modo da ridurre la loss nelle iterazioni successive.
- L'algoritmo di **discesa del gradiente** è il più comune:
 - Ogni peso viene spostato nella **direzione opposta al gradiente**, con un passo controllato dal learning rate $\eta \in \mathbb{R}$.

$$w_i = w_i - \eta \frac{\partial L}{\partial w_i}$$

nuovo peso = peso attuale - learning rate × gradiente

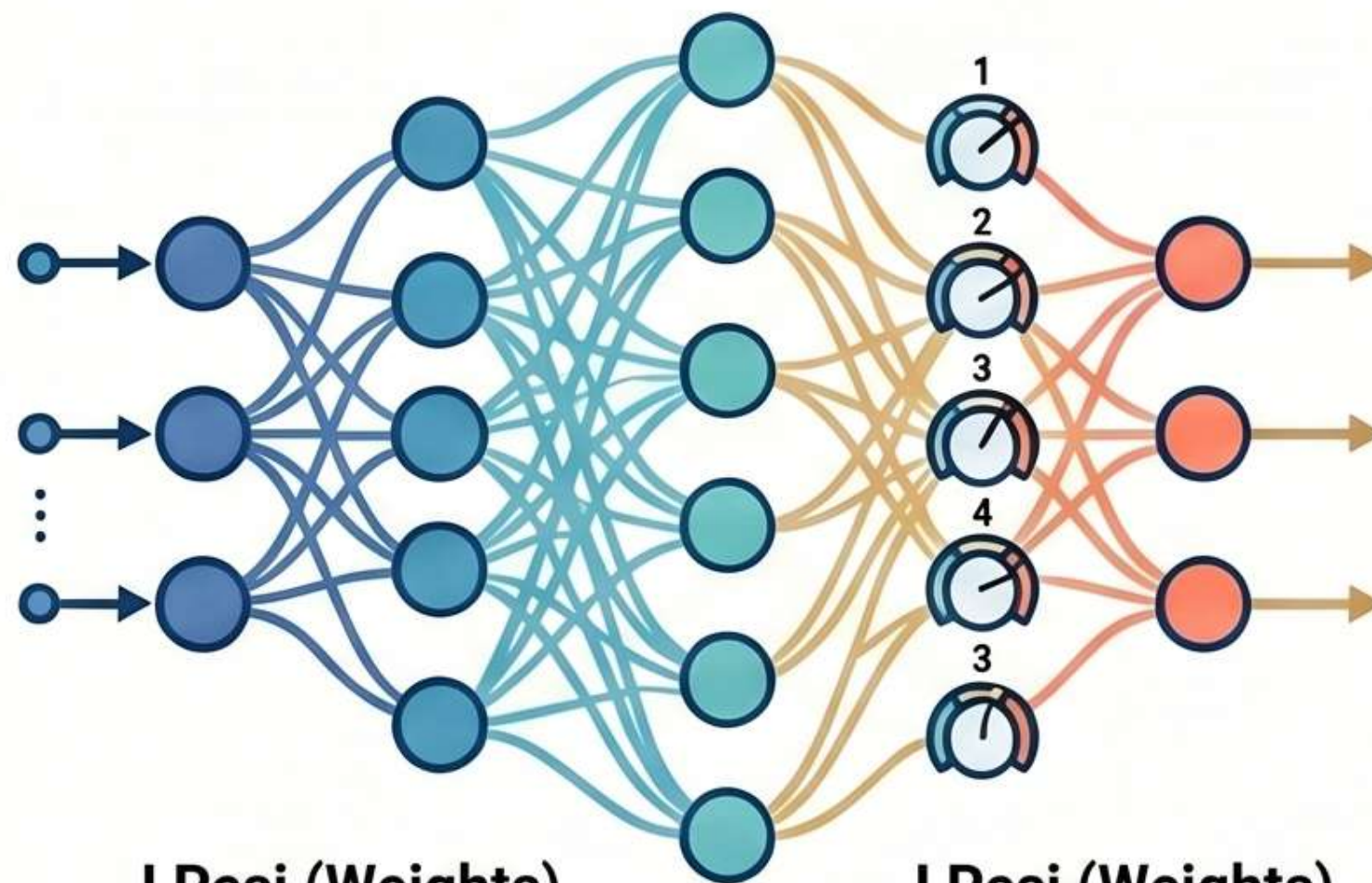
Come Imparano le Reti Neurali: Dall'Architettura all'Addestramento

Illustrare il funzionamento strutturale di una rete neurale e le fasi iterative del processo di addestramento (training).

Architettura e Componenti Base

Layer e Neuroni Artificiali

Unità interconnesse organizzate in livelli che trasformano l'input tramite funzioni non lineari.



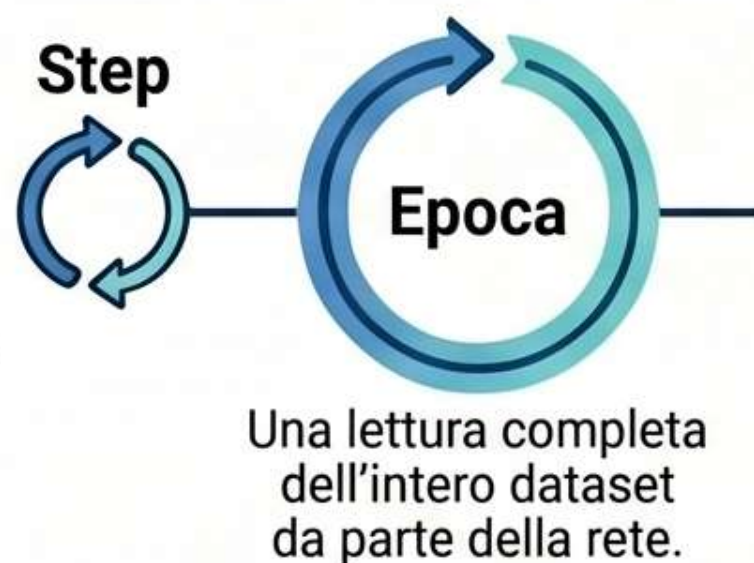
I Pesì (Weights)

Parametri numerici regolabili che determinano l'influenza di ogni connessione sulle prestazioni del modello.

I Pesì (Weights)

Parametri numerici regolabili che determinano l'influenza di ogni connessione sulle prestazioni del modello.

Terminologia temporale del processo di addestramento



Parallelismo

Tecnica necessaria per gestire modelli massivi su più GPU.

Forward Pass e Calcolo della Loss

Il modello genera una predizione e la confronta con il target per quantificare l'errore.



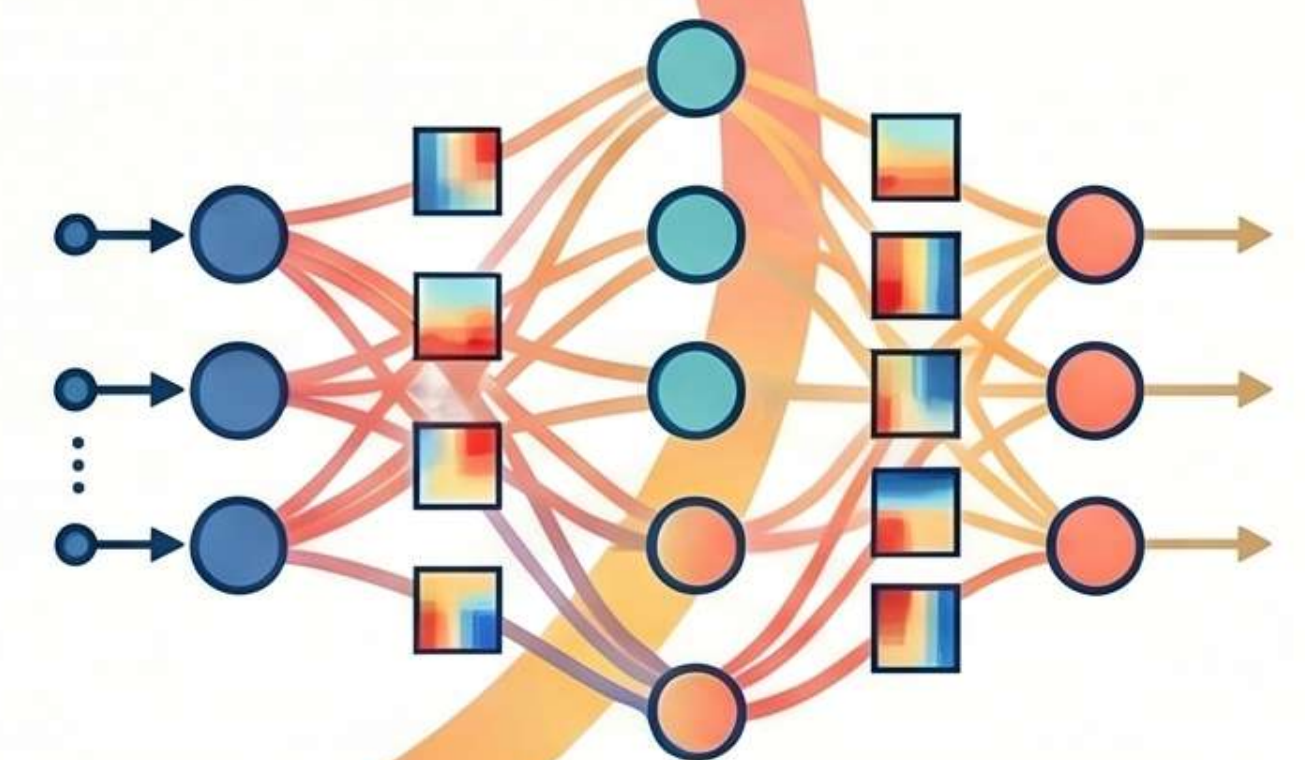
Terminologia temporale di addestramento

Una singola iterazione su un batch (sottoinsieme) di dati.



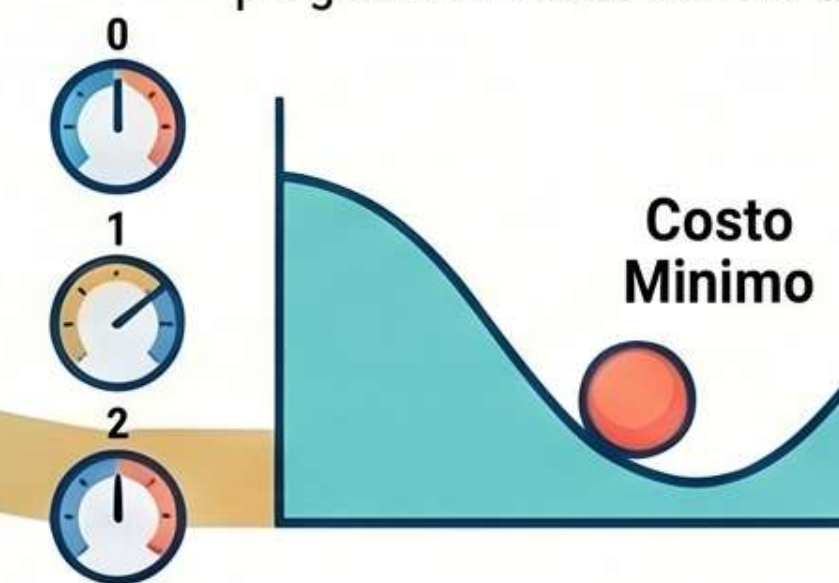
Il Ciclo Iterativo di Training

Un processo continuo che alterna Forward Pass e Backward Pass, aggiornando i pesi per ridurre progressivamente l'errore complessivo.



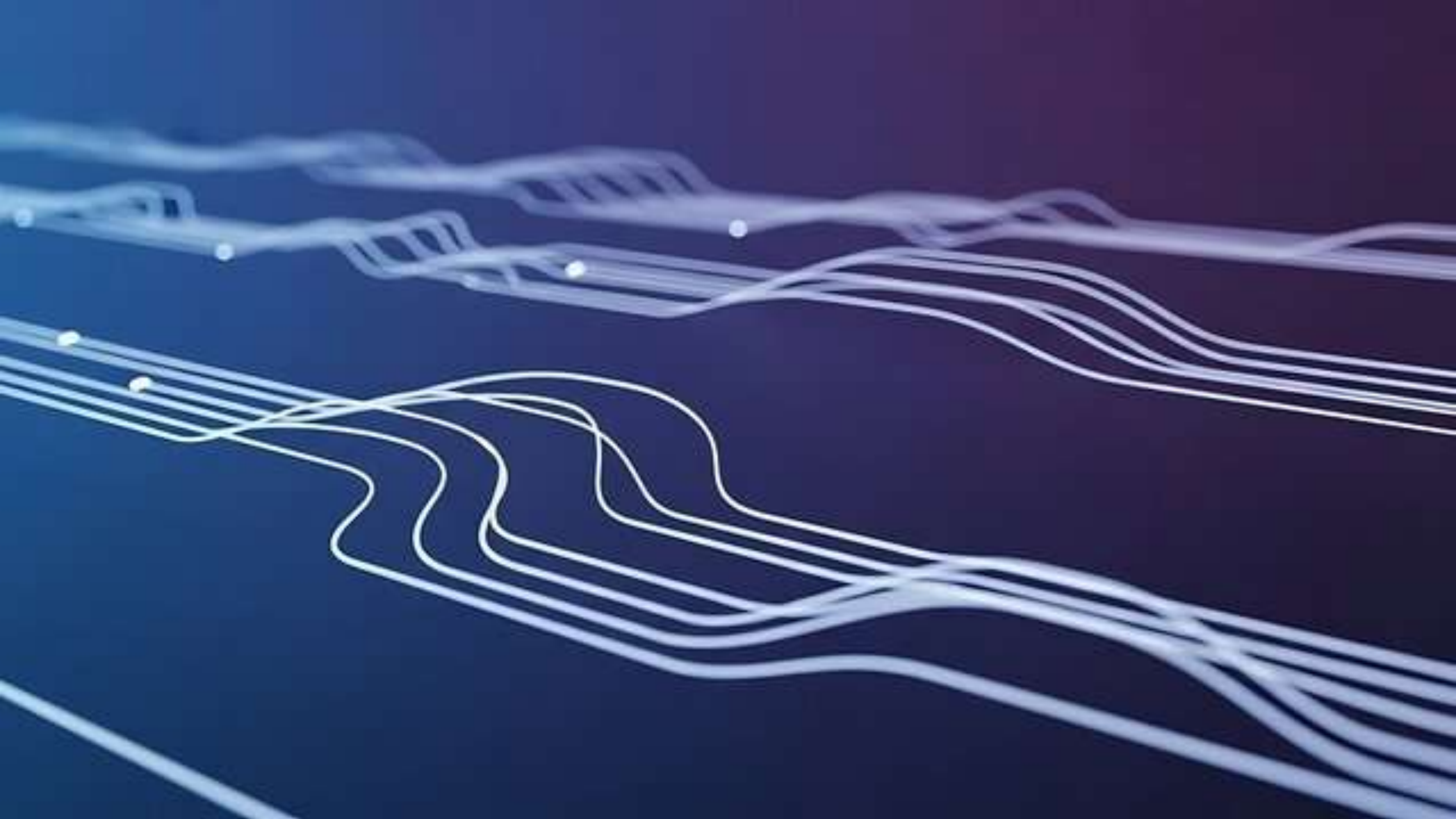
Backward Pass e Gradienti

L'errore viene propagato all'indietro per calcolare quanto ogni peso ha contribuito allo sbaglio.



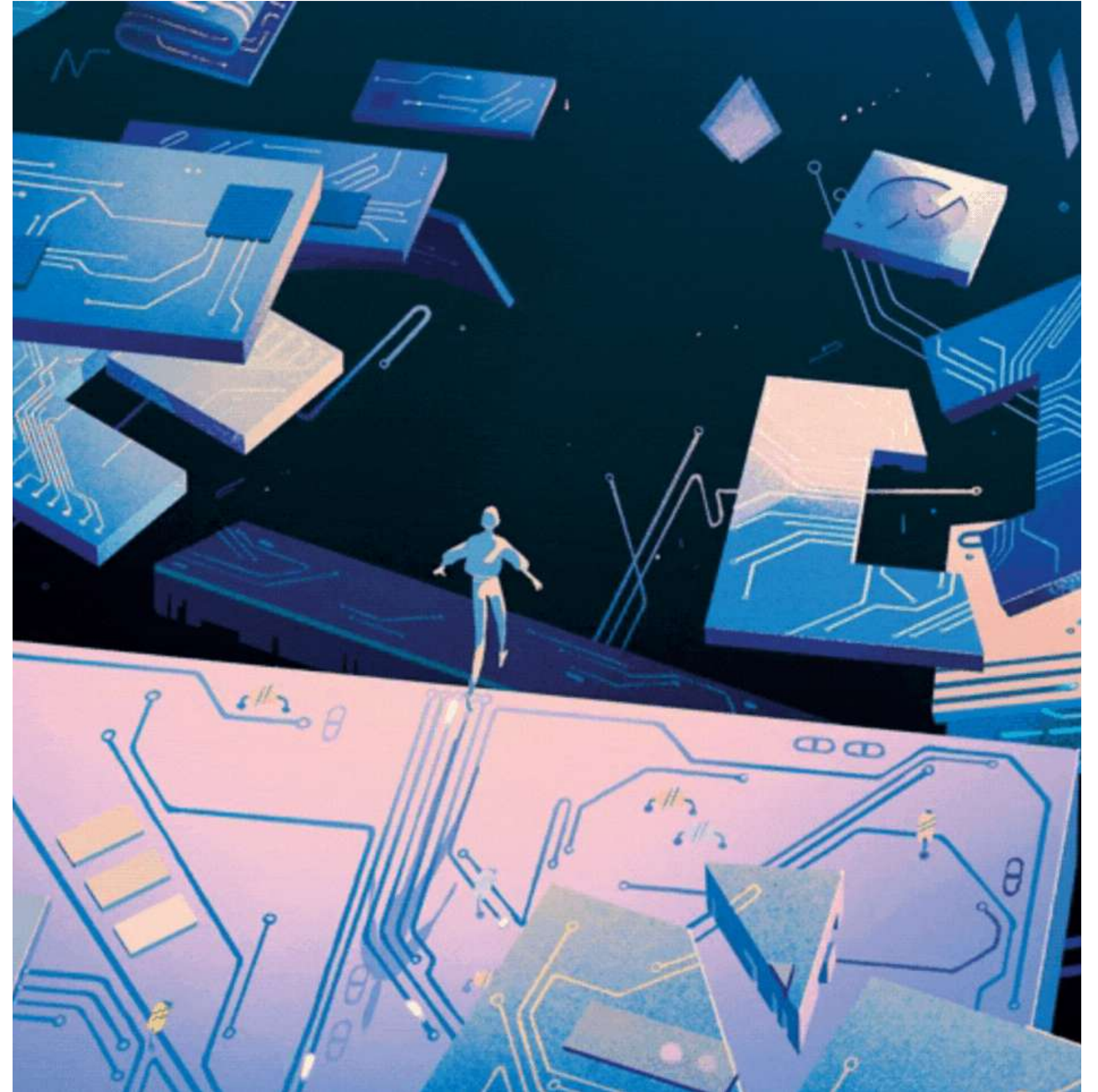
Discesa del Gradiente

I pesi vengono aggiornati usando il learning rate per minimizzare progressivamente l'errore complessivo.



Addestramento Parallelo di Reti Neurali

Motivazioni e Opportunità



Addestramento Parallelo

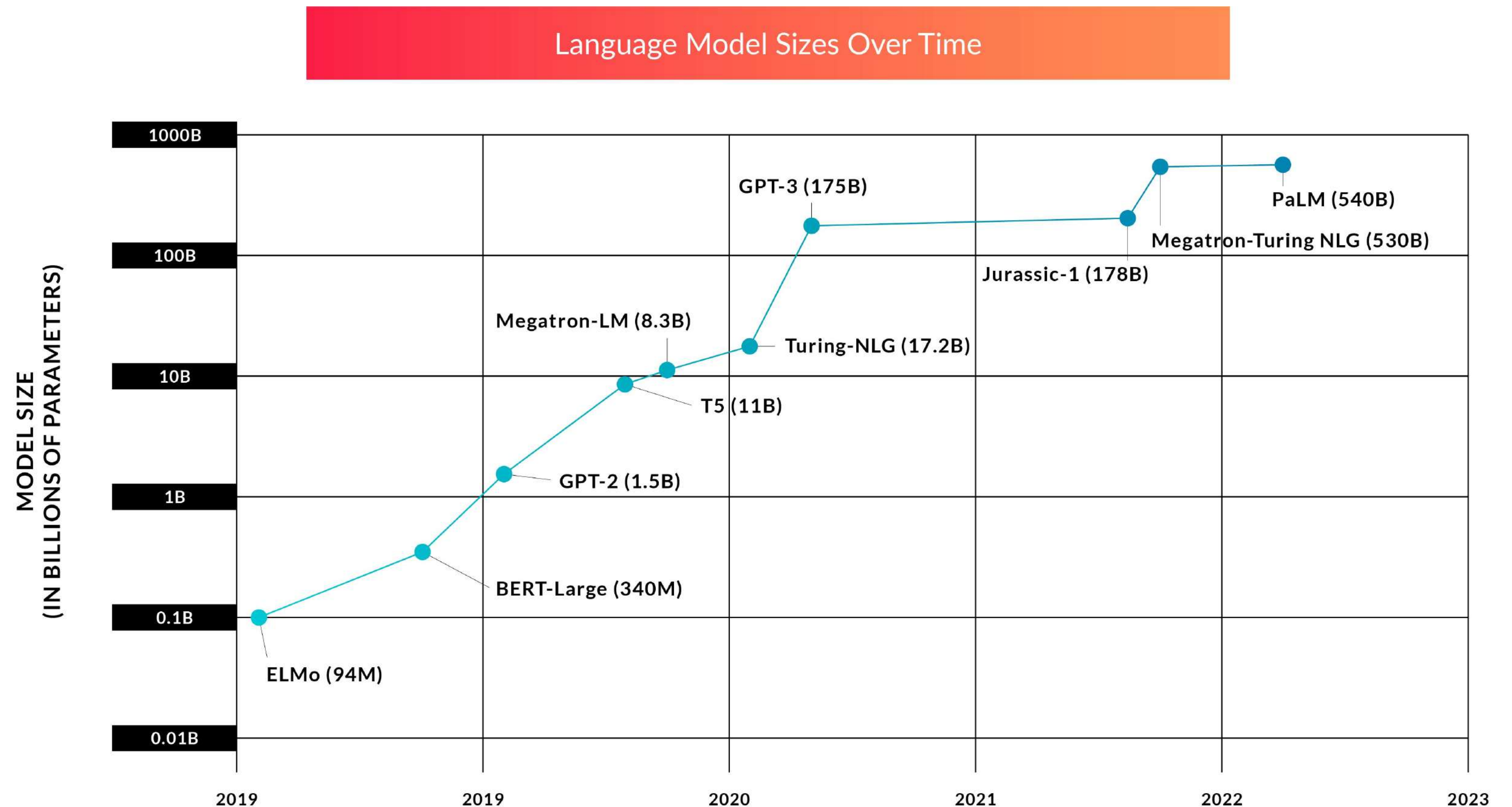
Motivazione

1. L'addestramento dei modelli moderni di intelligenza artificiale è diventato un problema computazionale troppo grande per essere affrontato efficacemente su una singola macchina.
2. Quando il problema diventa intrattabile per una singola macchina?
 1. Cosa succede quando il problema è troppo grande per una singola macchina?

Addestramento Parallelo

Motivazione

- Crescita della dimensione dei modelli
- Crescita dei dataset



Addestramento Parallelo

Motivazione

- Crescita della dimensione dei modelli
- Crescita dei dataset
- Tutto ciò si traduce in
 1. Aumento del costo computazionale del training
 2. Limiti di memoria delle singole GPU

Addestramento Parallelo

L'addestramento è computazionalmente costoso

- Come visto
 - **L'addestramento** non è una singola computazione, ma **un processo iterativo**.
 - Ogni iterazione richiede **molte operazioni matematiche**.
 - Il risultato di **una iterazione dipende dalla precedente**
- Ogni epoca passa l'intero dataset alla rete neurale
 - Dataset grandi richiedono **più tempo di elaborazione e più memoria**
 - Reti neurali grandi richiedono **più tempo di elaborazione e più memoria**

Addestrare più a lungo può migliorare le prestazioni, ma aumenta il costo.

Addestramento Parallelo

L'addestramento è computazionalmente costoso

- Perché il training richiede così tante risorse?
- Durante il training, il modello deve ripetere migliaia o milioni di batch di dati le stesse operazioni:
 - Leggere un batch di dati
 - Eseguire una forward pass
 - Calcolare l'errore
 - Eseguire una backward pass
 - Aggiornare i parametri

Addestramento Parallelo

Limiti del training su una singola GPU o CPU

- Una singola macchina può diventare insufficiente per tre motivi principali:
 - **Tempo di calcolo:** Il training richiede giorni, settimane o mesi
 - **Memoria:** Il modello, i gradienti e gli stati dell'ottimizzatore non entrano in una sola GPU
 - **Throughput dei dati:** La macchina non riesce ad alimentare abbastanza velocemente il dispositivo di calcolo (Bottleneck CPU/GPU)

Addestramento Parallelo

Limiti del training su una singola GPU o CPU

- **Throughput dei dati:** La macchina non riesce ad alimentare abbastanza velocemente il dispositivo di calcolo (Bottleneck CPU/GPU)
- Alcune tecniche pratiche per massimizzare il throughput dei dati:
 - Selezionare una **taglia del batch ottimale**: cercando di massimizzare l'efficienza
 - **Accumulo dei gradienti**: simulando l'utilizzo di una batch size che altrimenti non entrerebbe in memoria
 - **Addestramento distribuito o parallelo**: utilizzare più nodi processando più dati contemporaneamente

Addestramento Parallelo

Cosa occupa memoria durante il training?

- Un modello da n parametri non richiede memoria solo per n parametri, durante il training dobbiamo conservare:
 - Parametri del modello
 - Gradienti
 - Stati dell'algoritmo di ottimizzazione
 - Batch di input
 - Buffer temporanei

Addestramento Parallelo

Crescono dataset, modelli e costi

- Negli ultimi anni sono cresciuti contemporaneamente:
 - Dimensione dei dataset
 - Dimensione dei modelli
 - Numero di parametri
- Numero di GPU usate
- Costo energetico
- Costo economico
- Complessità delle infrastrutture

Addestramento Parallelo

Come possiamo ridurre i tempi?

- Il parallelismo è una risposta naturale
- L'idea è distribuire il lavoro:
 - Invece di usare **1 solo dispositivo**, usiamo più dispositivi che lavorano insieme durante l'addestramento



Tipologie di Addestramento Parallelo

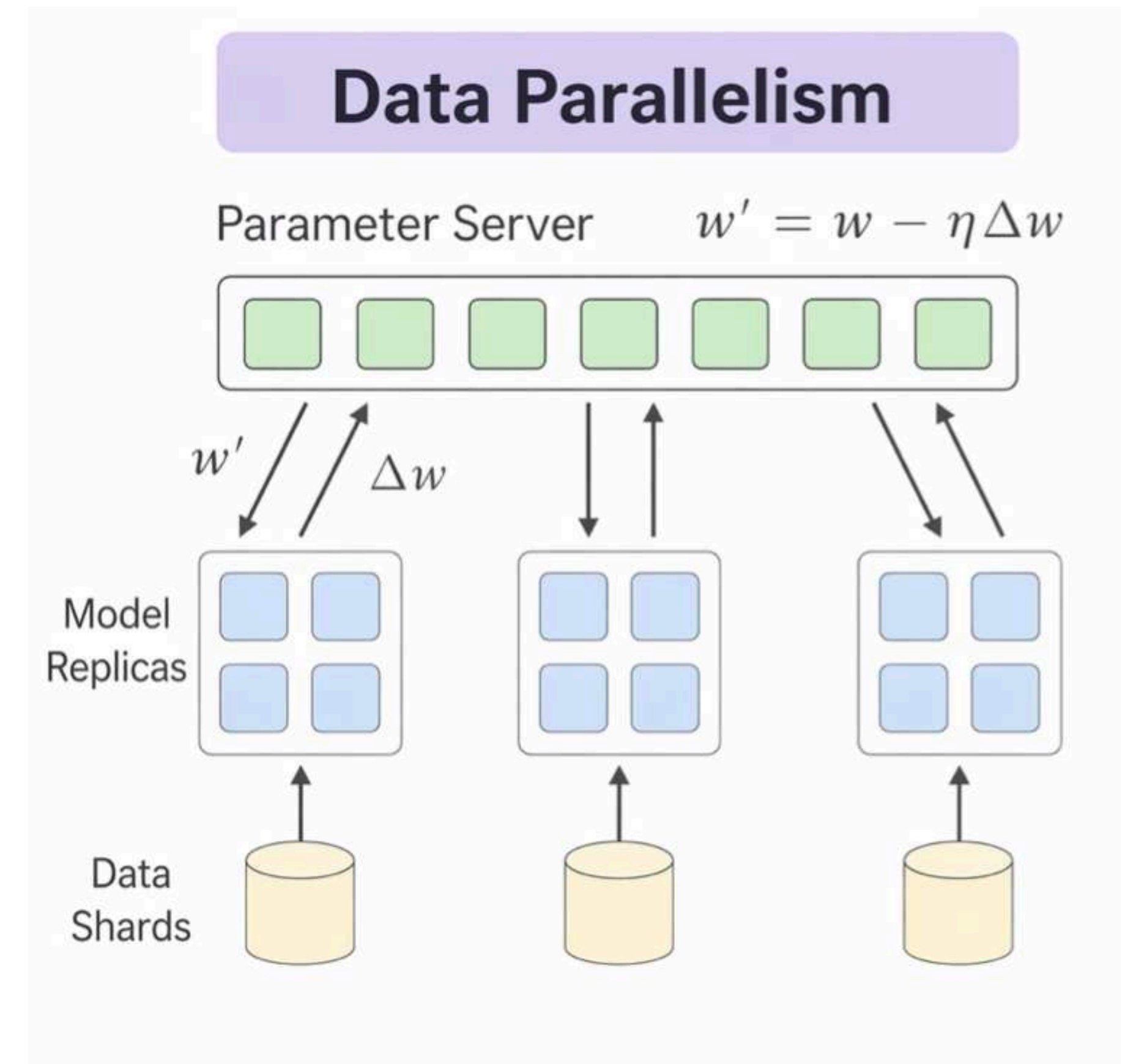
- Data Parallelism
- Model Parallelism
- Pipeline Parallelism
- Tensor Parallelism



Data Parallelism

Idea

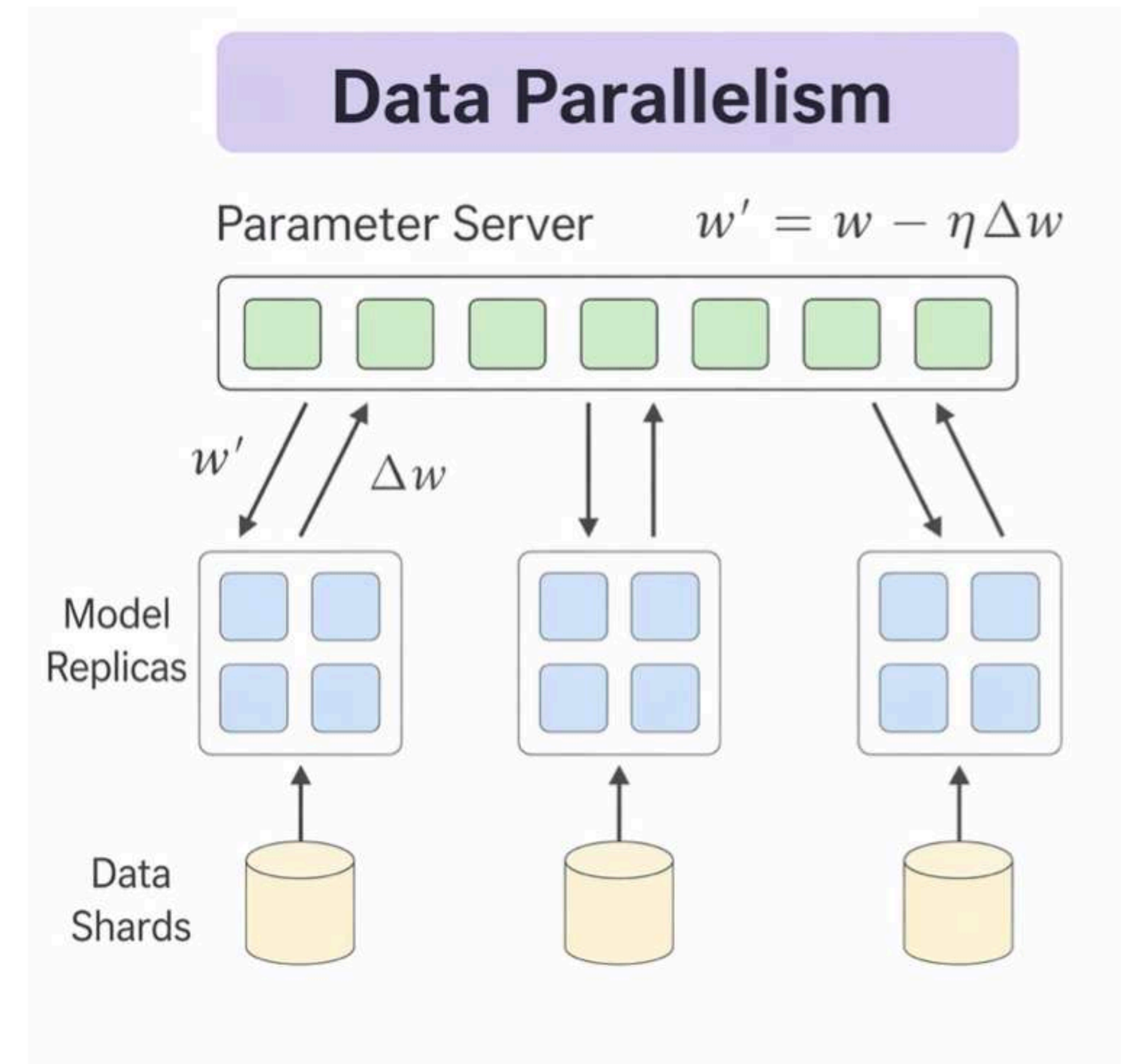
- La strategia di training distribuito più intuitiva
- Ogni nodo/worker/GPU
 - Ha una copia completa del modello
 - Riceve una porzione diversa del dataset (batch).
 - Calcola i gradienti locali
 - Li condivide per aggiornare i parametri globali del modello



Data Parallelism

Costi e limiti

- il modello deve stare interamente nella memoria di ogni GPU;
- la sincronizzazione dei gradienti può dominare il tempo;
- aumentando il numero di GPU, la comunicazione cresce;
- batch size globale troppo grande può peggiorare la convergenza;
- stragglers: se un nodo è lento, tutti aspettano.
- Tempo per step $\approx$ tempo di calcolo + tempo di comunicazione + tempo di sincronizzazione



Data Parallelism

Speedup di uno step

- Esempio:

1 GPU
100 ms calcolo

4 GPU
30 ms calcolo + 20 ms comunicazione

$$S_{dp} = \frac{100\text{ms}}{30\text{ms} + 20\text{ms}} = 2.0$$

Speedup reale = 2x $\neq$ 4x

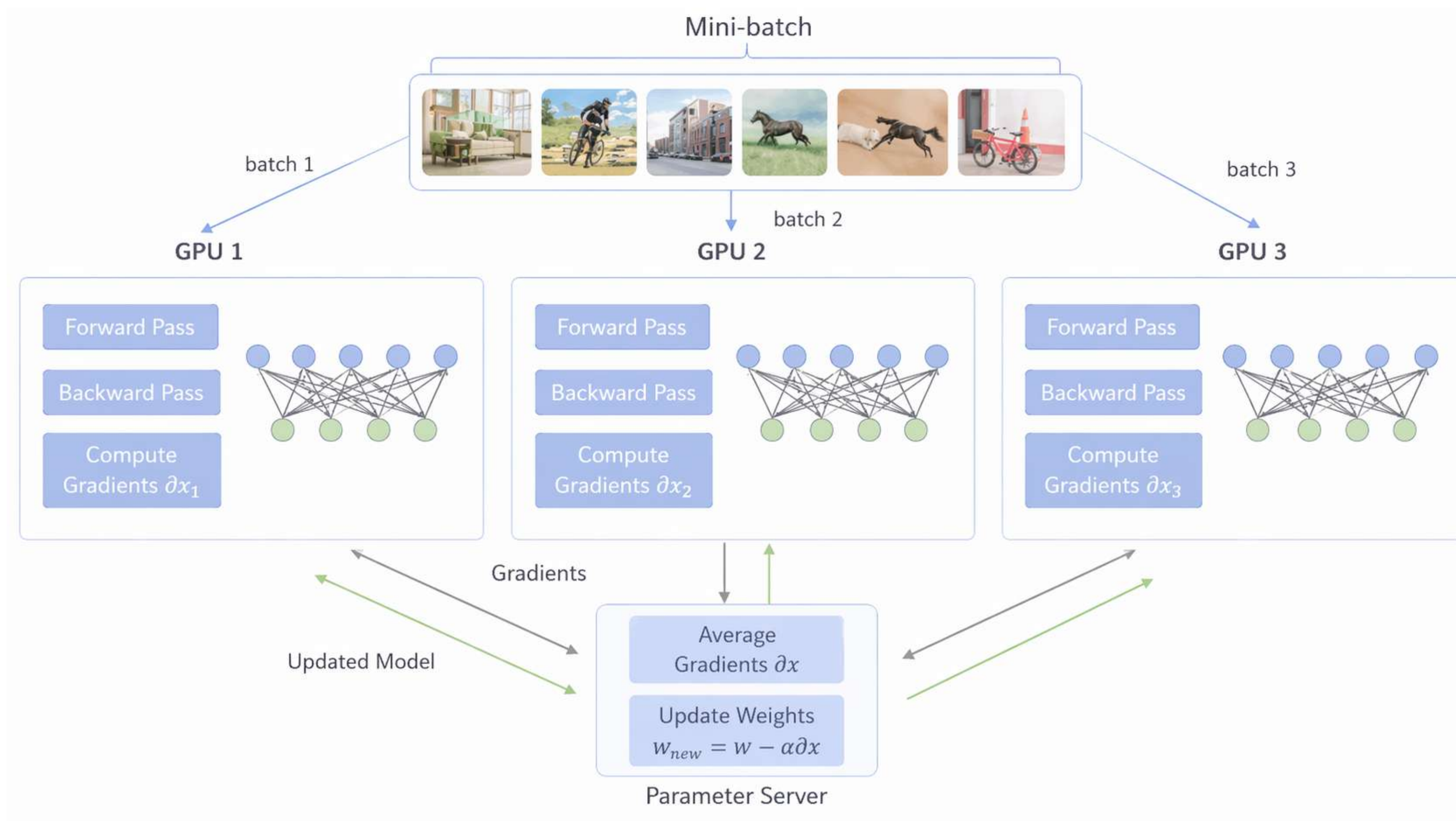
Data Parallelism

Implementazione 1) Parameter Server

- Il **parameter server** è una possibile implementazione del data parallelism
- Distinguiamo due ruoli:
 - *Worker* - Che eseguono gli step di addestramento e producono gradienti locali
 - *Server* - Incaricato di mantenere e aggiornare i parametri globali del modello

Data Parallelism

Parameter Server



Data Parallelism

Implementazione 2) > Approccio Decentralizzato

- Un'alternativa al parameter server è l'**approccio decentralizzato**
- Tutti i processi partecipano direttamente alla sincronizzazione dei gradienti
 - Non esiste un nodo centrale
 - Ogni nodo calcola i gradienti globali e partecipa alla riduzione collettiva per combinare tutti i worker

Data Parallelism

Problemi e limiti

- Se il modello non entra nella memoria di una singola GPU, il data parallelism basta?
- No, perché ogni GPU deve comunque contenere una copia completa del modello.
 - La soluzione?

Data Parallelism

Problemi e limiti

- Se il modello non entra nella memoria di una singola GPU, il data parallelism basta?
- No, perché ogni GPU deve comunque contenere una copia completa del modello.
- Model Parallelism -> Parallelizziamo il modello

Model Parallelism

Idea

- Invece di replicare il modello e dividere il dataset
- Si divide il modello tra più dispositivi.

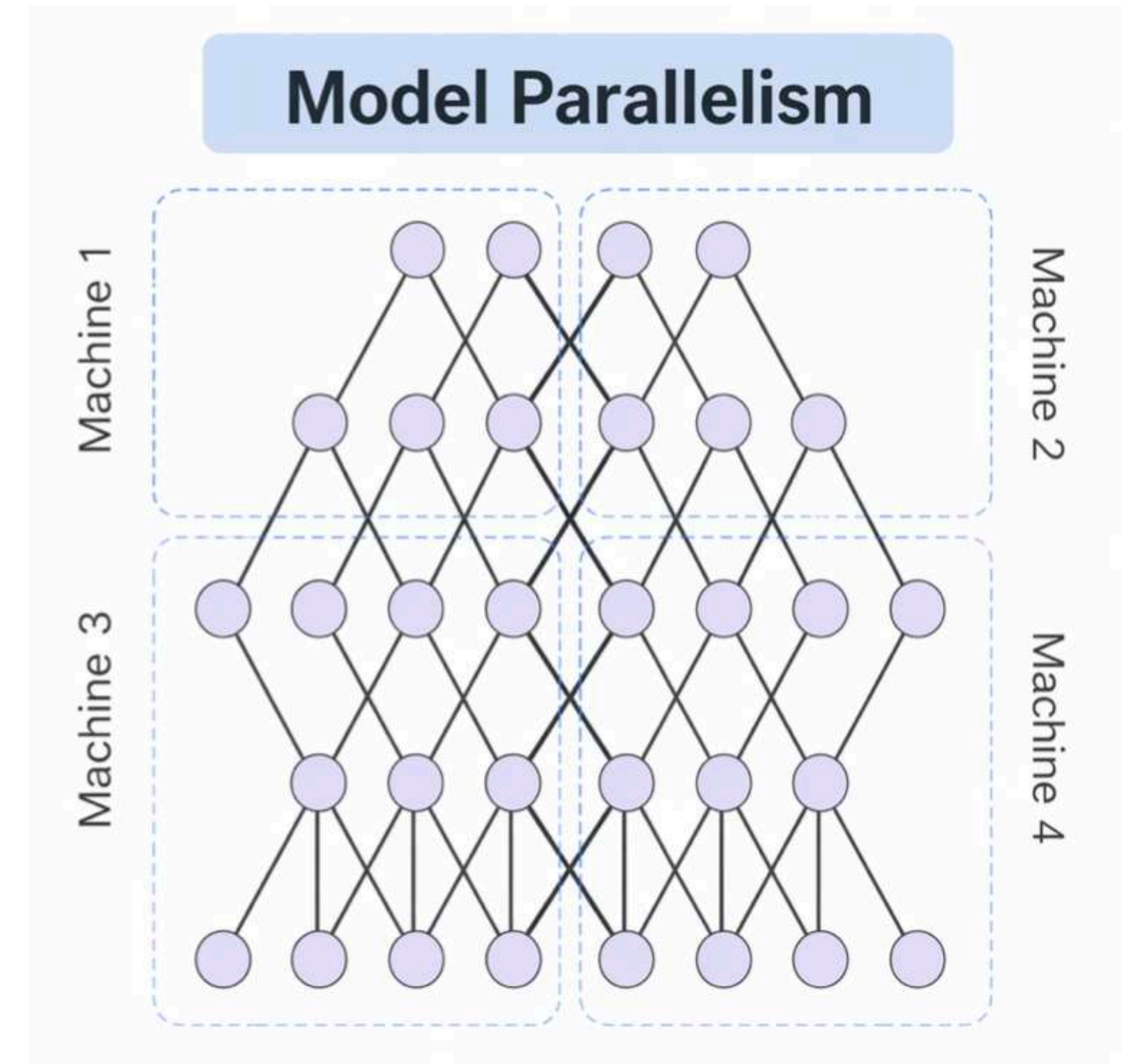
Esempio:

Machine 1: Layer 1–10

Machine 2: Layer 11–20

Machine 3: Layer 21–30

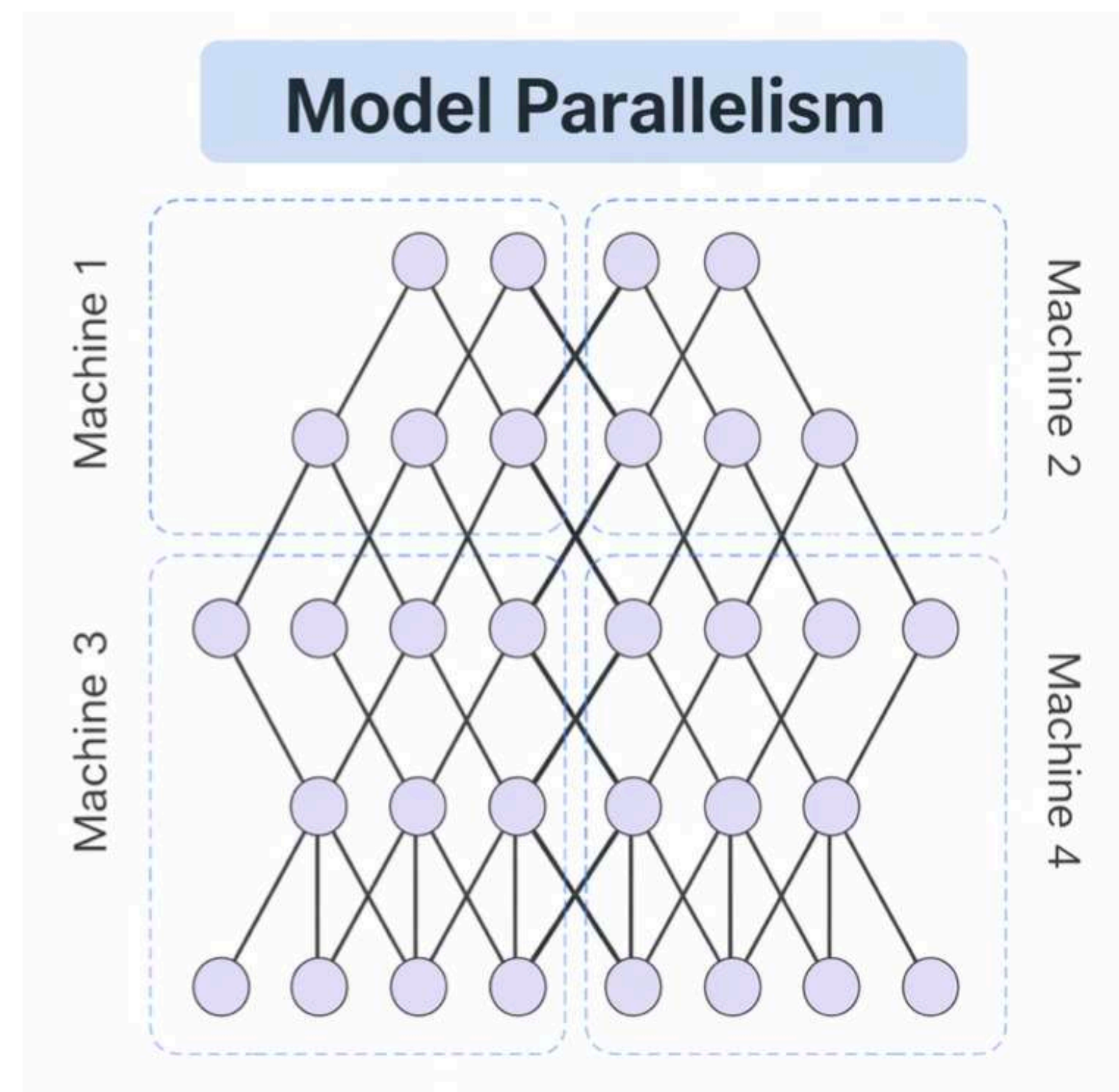
Machine 4: Layer 31–40



Model Parallelism

Quando è utile?

- modello troppo grande per una singola GPU;
- LLM con miliardi o trilioni di parametri;
- memoria insufficiente per pesi, gradienti, stati dell'ottimizzatore e attivazioni.



Model Parallelism

Problema

- Il worker 1 deve aspettare l'output del worker 0
- Il worker 2 deve aspettare l'output del worker 1
- ...
- Il worker i deve aspettare l'output del worker $i - 1$

Model Parallelism

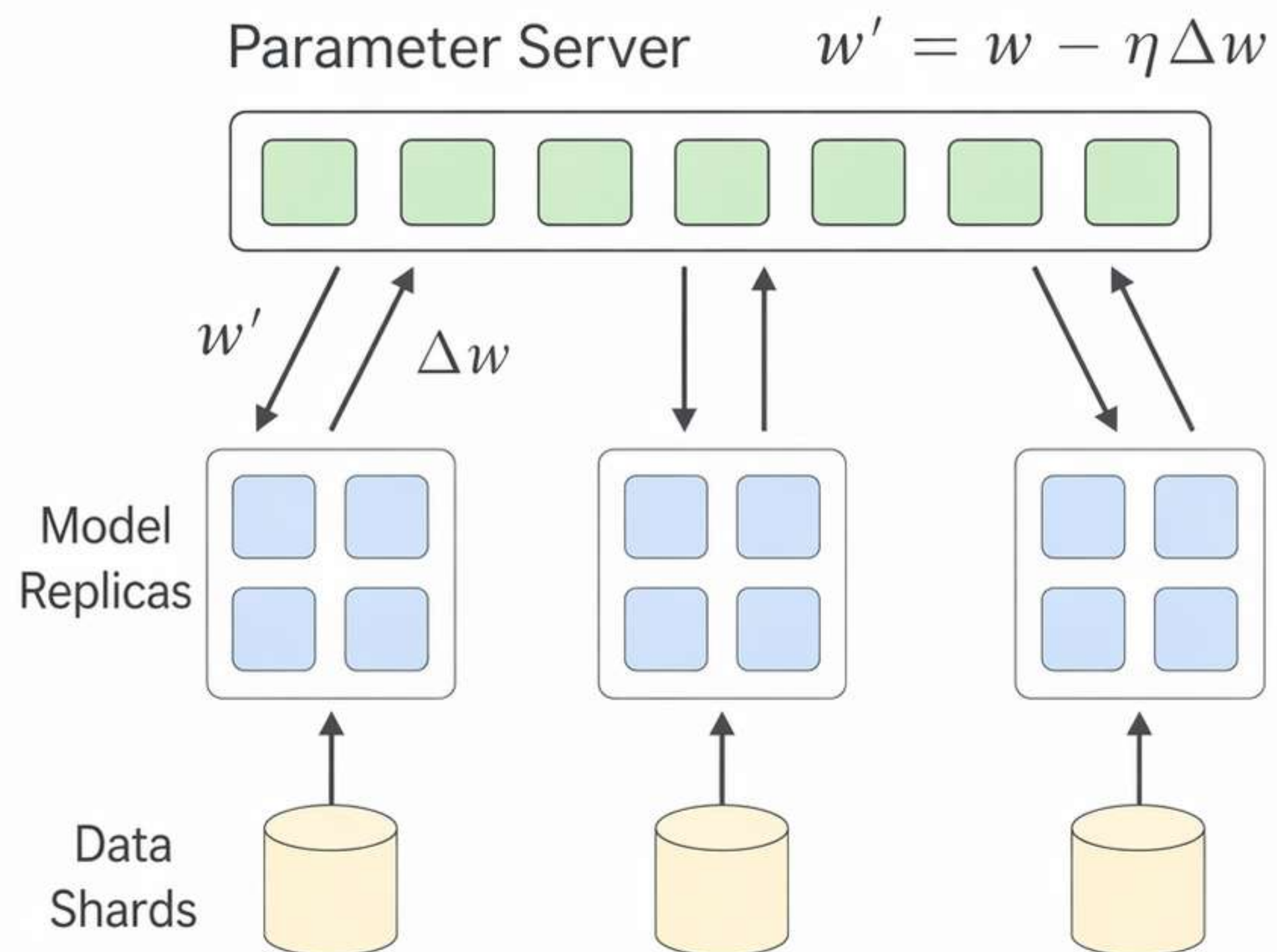
Problema

- Quindi non basta “spezzare il modello”: bisogna gestire comunicazione e pipeline.
- Il model parallelism può introdurre un problema di gestione inefficiente delle risorse
- Molti nodi attendono in *idle* il loro turno

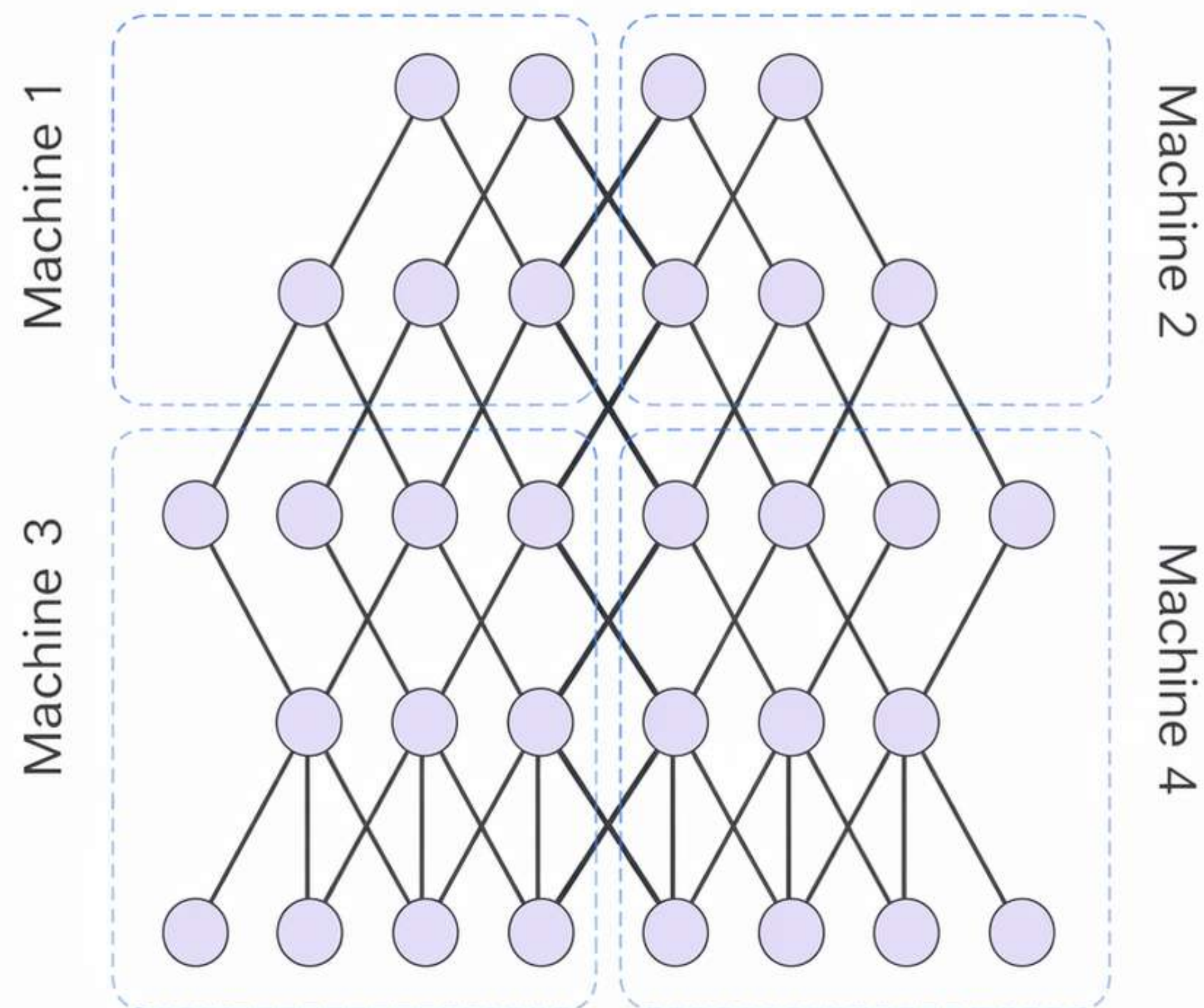
	Istante 1	Istante 2	Istante 3	Istante 3
Worker 0	MB1	Idle	Idle	Idle
Worker 1	Idle	MB1	Idle	Idle
Worker 2	Idle	Idle	MB1	Idle
Worker 3	Idle	Idle	Idle	MB1

Le due architetture a confronto

Data Parallelism

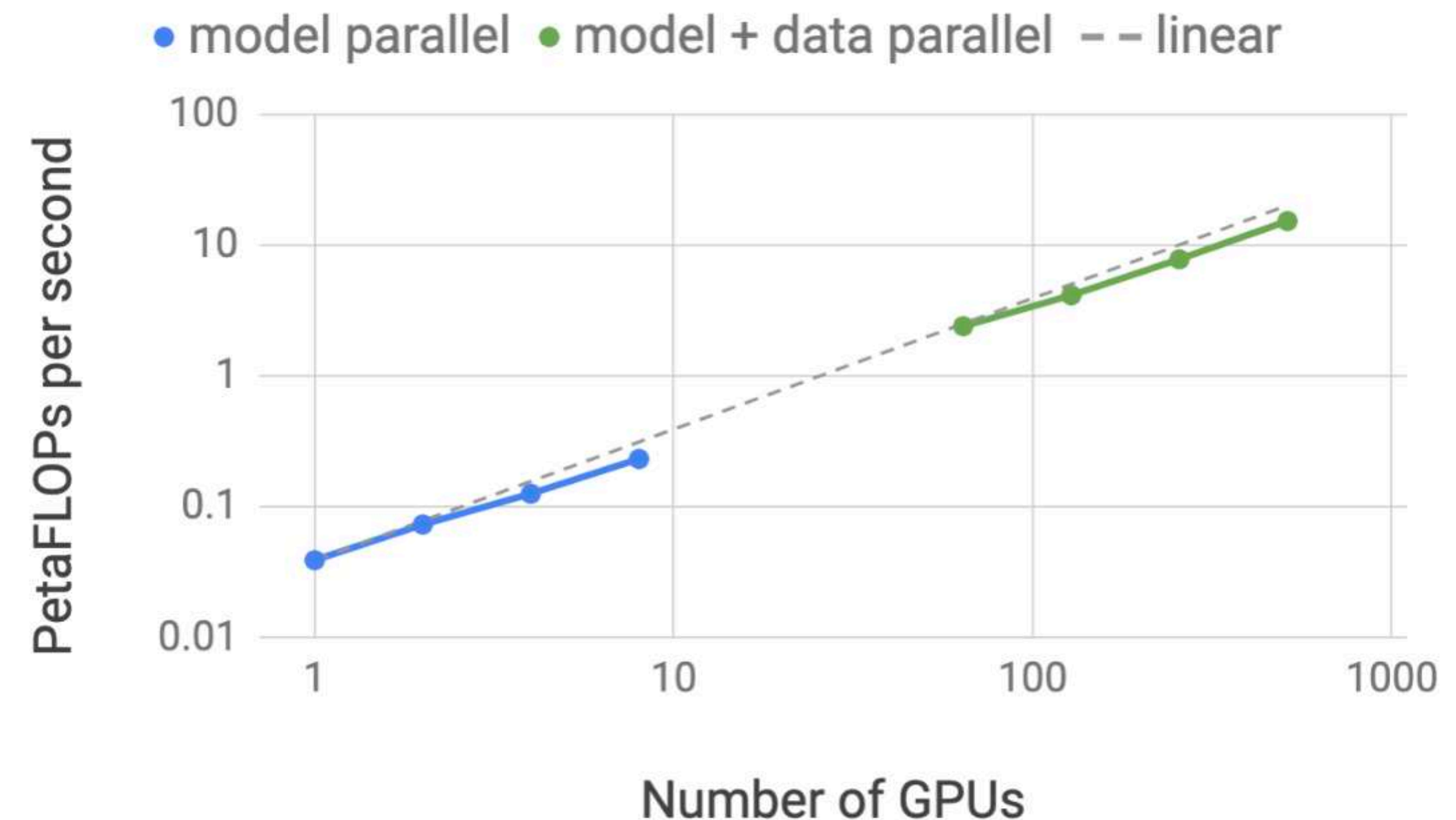


Model Parallelism



Le due architetture a confronto

- Queste due soluzioni **non sono mutualmente esclusive**
 - Risolvono due problemi differenti:
 - Data Parallelism > Dati troppo grandi
 - Model Parallelism > Modello Troppo Grande
 - Possono essere combinate per costruire soluzioni ibride



[1] Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism

Addestramento Parallelo

Da MPI ai Modelli Distribuiti

- Il training distribuito è un'applicazione concreta dei concetti studiati in questo corso
 - Processi MPI -> Worker / processo di training
 - Nodo del cluster -> Macchina con una o più GPU
 - Messaggi -> batch, gradienti, parametri
 - Reduce / Allreduce -> Media dei gradienti
 - Speedup -> Riduzione del tempo di training
 - Overhead -> Comunicazione e sincronizzazione



Pipeline Parallelism

Idea

- Il principale problema del **model parallelism** è la gestione inefficiente delle risorse:
 - I nodi possono rimanere in attesa degli altri nodi
- Nel **pipeline parallelism**
 - il modello è ancora diviso tra più GPU
 - Ma il batch viene ulteriormente diviso in micro-batch.
 - Ogni micro-batch attraversa gli stadi della pipeline.

Pipeline Parallelism

Idea

- Il pipeline parallelism non è semplicemente “dividere il modello”
 - Il flusso è organizzato in micro-batch per tenere occupati più dispositivi contemporaneamente.
- Dividere il modello in stadi e far fluire micro-batch attraverso la pipeline.

T=0

T=1

T=2

T=3

T=4

T=5

Micro-batch 1 : Worker 0 → Worker 1 → Worker 2 → Worker 3

Micro-batch 2 : Worker 0 → Worker 1 → Worker 2 → Worker 3

Micro-batch 3 : Worker 0 → Worker 1 → Worker 2 → Worker 3

Pipeline Parallelism

Differenze con Model Parallelism

MB = Micro Batch

Model Parallelism

	Istante 1	Istante 2	Istante 3	Istante 3
Worker 0	MB1	Idle	Idle	Idle
Worker 1	Idle	MB1	Idle	Idle
Worker 2	Idle	Idle	MB1	Idle
Worker 3	Idle	Idle	Idle	MB1

Pipeline Parallelism

	Istante 1	Istante 2	Istante 3	Istante 3
Worker 0	MB1	MB2	MB3	MB4
Worker 1	Idle	MB1	MB2	MB3
Worker 2	Idle	Idle	MB1	MB2
Worker 3	Idle	Idle	Idle	MB1

Tensor Parallelism

Tensori

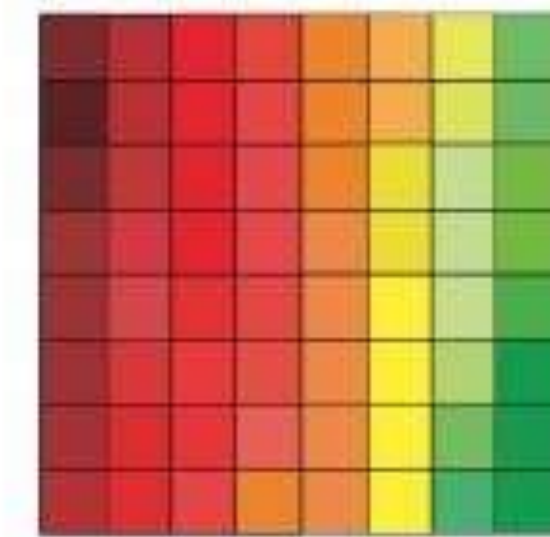
- Cosa sono i tensori?
 - Un tensore è una **struttura dati multidimensionale** usata per rappresentare numeri, dati e parametri nei modelli di deep learning.
 - *Scalare* → 0 dimensioni → singolo numero
 - *Vettore* → 1 dimensione → lista di valori
 - *Matrice* → 2 dimensioni → tabella
 - ...
 - *Tensore* → N dimensioni → dati multidimensionali

vector



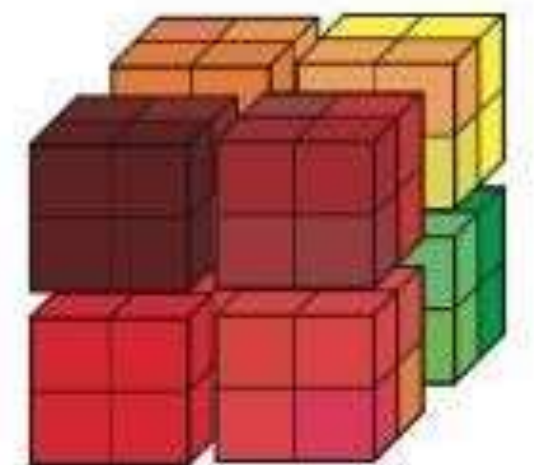
$$x \in \mathbb{R}^d$$

matrix



$$\mathbf{X} \in \mathbb{R}^{d_0 \times d_1}$$

tensor



$$\mathcal{X} \in \mathbb{R}^{d_0 \times \dots \times d_n}$$

Nel deep learning i tensori rappresentano principalmente: dati, parametri del modello e gradienti

Tensor Parallelism

Idea - Parallelizzare ad un Livello più Basso

- Le operazioni eseguite nel training sono rappresentate come operazioni su tensori, quindi...
 - Parallelizziamo sulle operazioni tensoriali, come ad esempio:
 - Somma tensoriale $\mathbf{Z} = \mathbf{X} + \mathbf{Y}$
 - Moltiplicazione elemento per elemento $\mathbf{Z} = \mathbf{X} \odot \mathbf{Y}$
 - Prodotto tensoriale $\mathbf{Z} = \mathbf{XY}$
- $\mathbf{X}$ e $\mathbf{Y}$ vengono divisa tra più GPU
- Ogni worker calcola una parte di $\mathbf{Z}$
- I risultati vengono combinati

Pipeline & Tensor Parallelism

Megatron-LM - scalare il training di modelli molto grandi

- Nel training di modelli molto grandi, tecniche come data, tensor e pipeline parallelism vengono spesso combinate;
- Megatron-LM (2022) mostra che:
 - forme differenti di parallelismo possano essere combinate per scalare a migliaia di worker e modelli con dimensioni estremamente grandi.

Megatron-LM - scalare il training di modelli molto grandi

Pipeline & Tensor Parallelism

- Megatron-LM combina:
 - Tensor Parallelism
 - Pipeline Parallelism
 - Data Parallelism
- Risultando in un approccio efficiente di **intra-layer model parallelism** per l'addestramento parallelo di *Large Language Model*

Memory optimization e stati dell'ottimizzatore

ZeRO

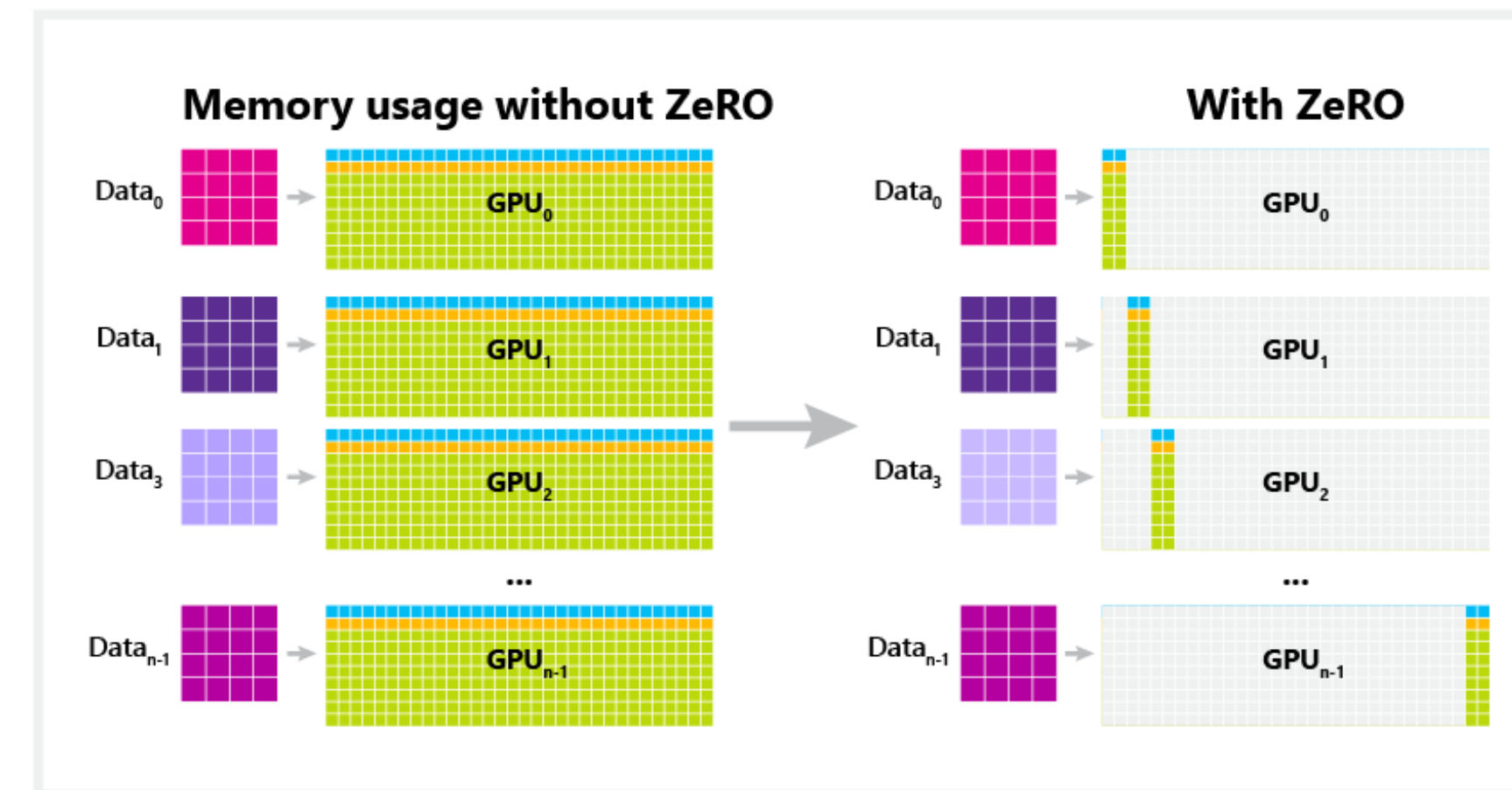
- Durante il training non bisogna memorizzare solo i pesi del modello.
- Per ogni parametro vengono tipicamente memorizzati:
 - Peso (valore numerico del parametro)
 - Gradiente
 - Stati dell'algoritmo di ottimizzazione
 - Attivazioni intermedie
 - Buffer temporanei
- La gestione di tutta questa memoria può diventare un problema

ZeRO

Memory optimization e stati dell'ottimizzatore

DeepSpeed + ZeRO

- **ZeRO** (2019), proposto da *Microsoft*
 - **Riduce la ridondanza della memoria** partizionando tra dispositivi:
 - stati del modello
 - gradienti
 - stati dell'ottimizzatore
 - invece di replicarli interamente su ogni worker.
- Invece, nel Data Parallel classico:
 - ogni GPU contiene pesi + gradienti + optimizer states



Scale

- 100B parameter
- 10X bigger

Speed

- Up to 5X faster

Cost

- Up to 5X cheaper

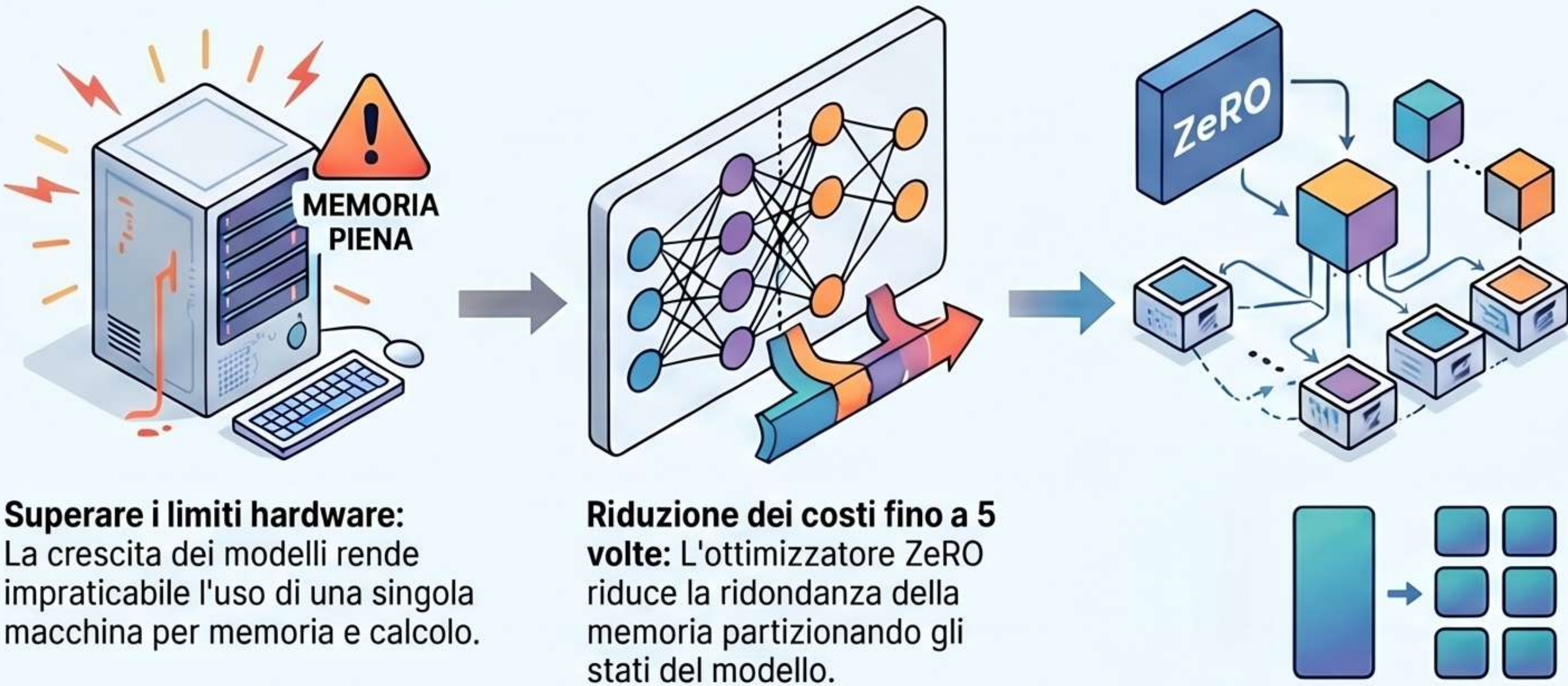
Usability

- Minimal code change

Addestramento Distribuito: Scalare l'Intelligenza Artificiale

L'addestramento di modelli AI moderni richiede risorse che superano la capacità di una singola macchina. Questa infografica descrive come il parallelismo dei dati, dei modelli e l'ottimizzazione della memoria permettano di distribuire il carico computazionale su cluster di GPU.

Le Sfide e la Soluzione ZeRO



Superare i limiti hardware:
La crescita dei modelli rende impraticabile l'uso di una singola macchina per memoria e calcolo.

Riduzione dei costi fino a 5 volte: L'ottimizzatore ZeRO riduce la ridondanza della memoria partizionando gli stati del modello.

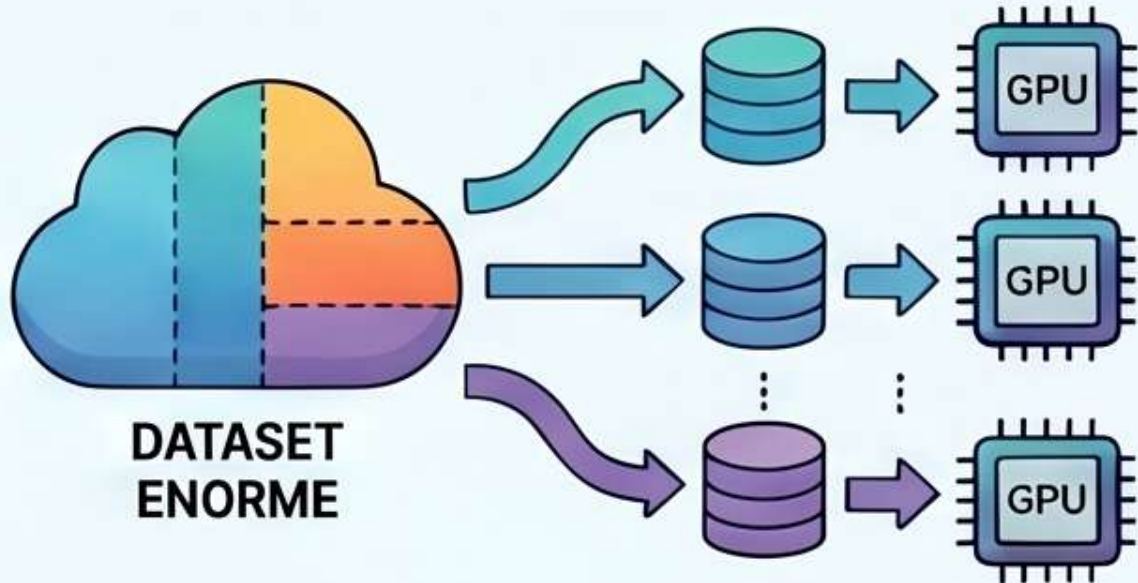
Sintesi dei vantaggi delle architetture distribuite

Strategia	Obiettivo Principale	Vantaggio Chiave
Parallelismo di Dati	Gestione Dataset	Throughput elevato
Parallelismo di Modello	Gestione Parametri	Supporto a modelli giganti
Ottimizzazione ZeRO	Efficienza Memoria	Ottimizzazione stati

Strategie di Parallelismo

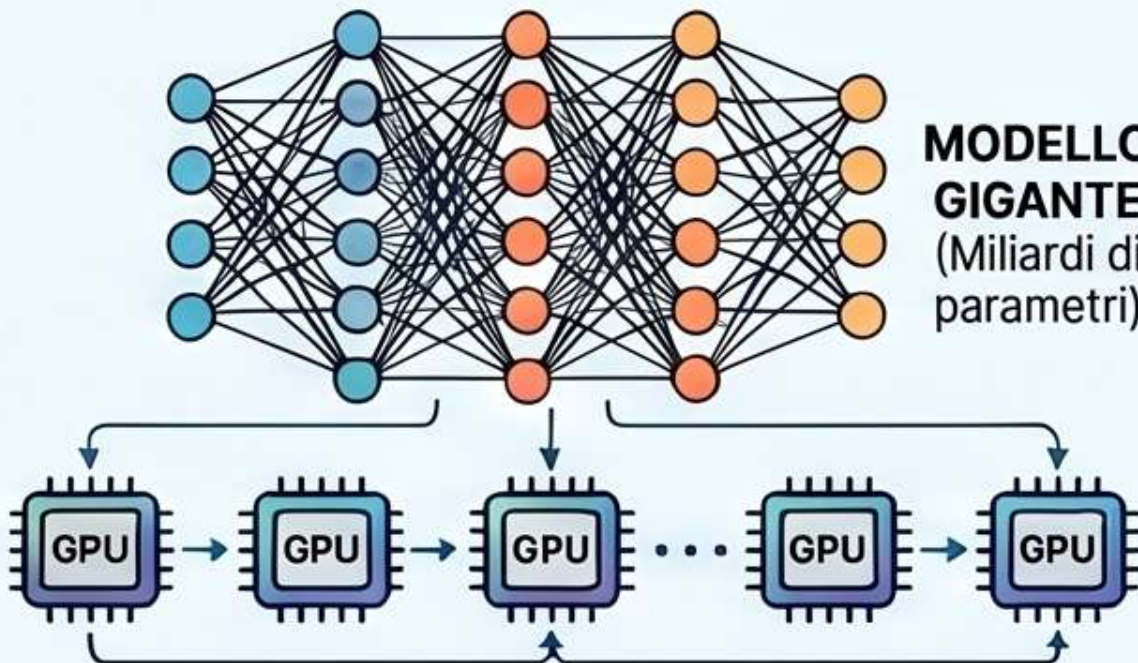
Data Parallelism

Divide dataset enormi tra vari nodi del cluster per l'elaborazione simultanea.



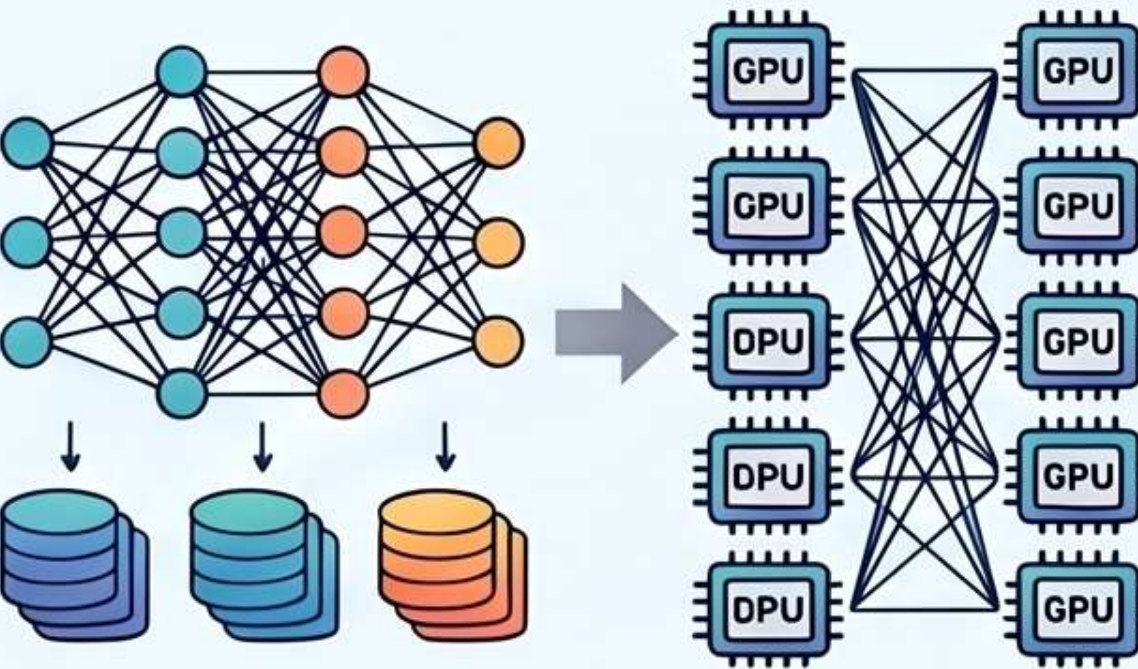
Model & Pipeline Parallelism

Scompone modelli con miliardi di parametri per adattarli alla memoria di più GPU.



Soluzioni Ibride (Megatron-LM)

Integra diverse forme di parallelismo per scalare l'addestramento su migliaia di worker.

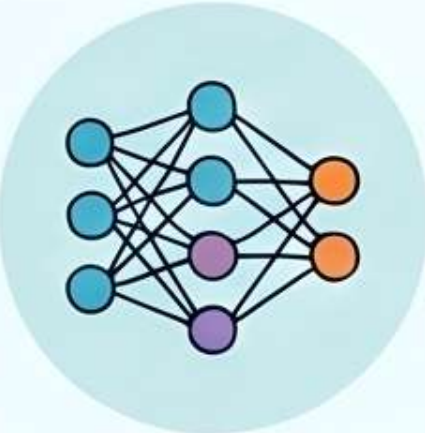




L'Addestramento delle Reti Neurali: Dal Singolo Neurone al Calcolo Distribuito

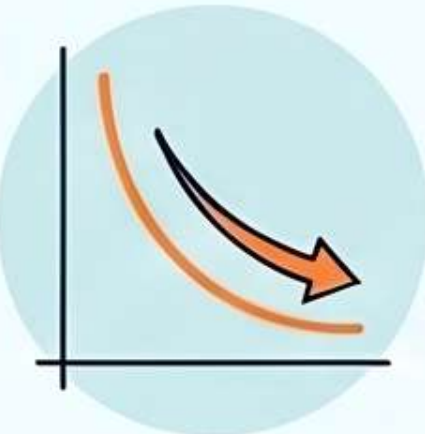
Le reti neurali sono sistemi di unità interconnesse che apprendono attraverso un processo iterativo di ottimizzazione dei pesi. Con la crescita esponenziale dei Large Language Models (LLM), l'addestramento distribuito su più GPU è diventato indispensabile per superare i limiti di memoria e tempo di calcolo.

IL CICLO DI ADDESTRAMENTO ITERATIVO



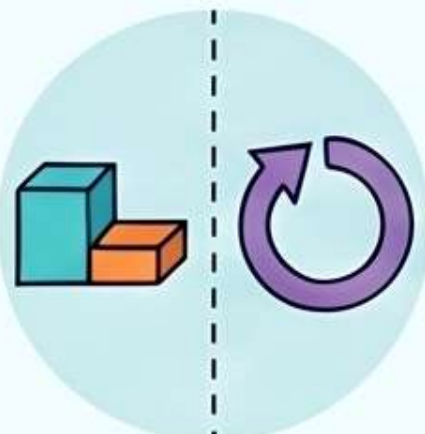
Forward & Backward Pass

L'input attraversa i layer (Forward), viene calcolato l'errore (Loss) e propagato all'indietro (Backward).



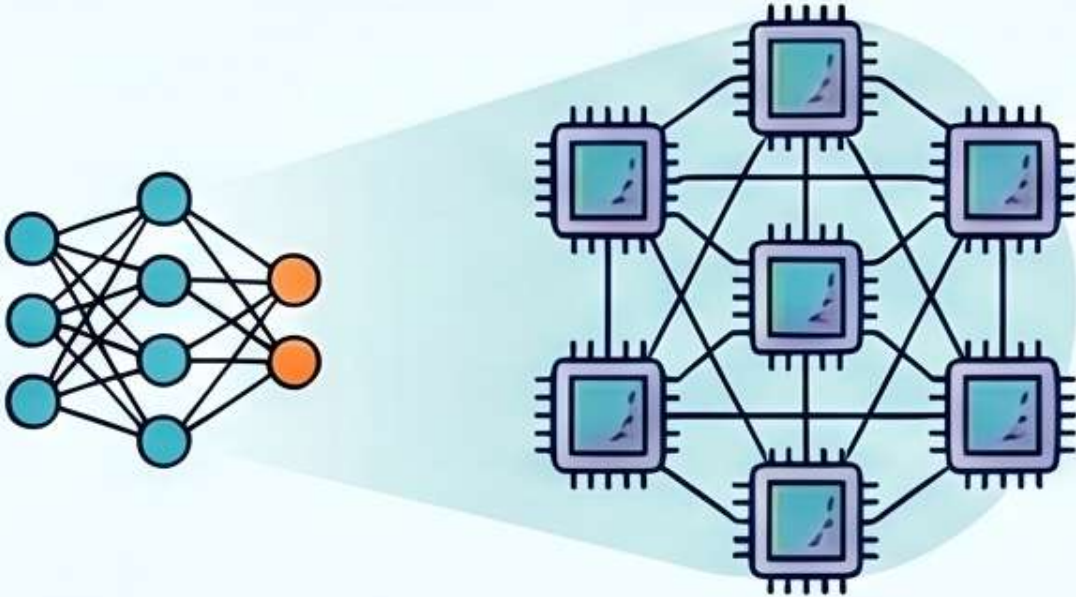
Ottimizzazione tramite Gradienti

Vengono calcolate le derivate parziali per determinare come variare ogni peso per ridurre l'errore.

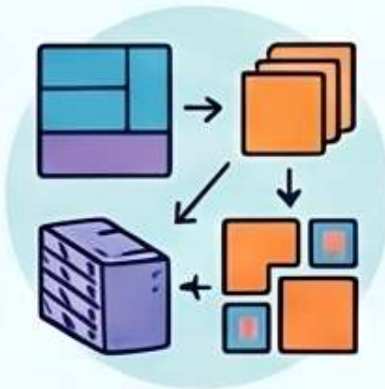
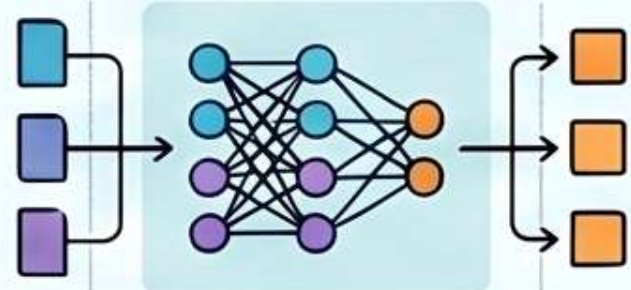
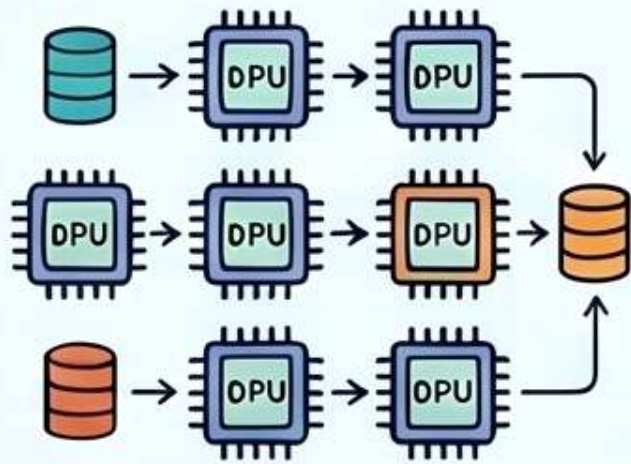


Step vs Epoca

Uno Step è un'iterazione su un batch; l'Epoca è la lettura dell'intero dataset.



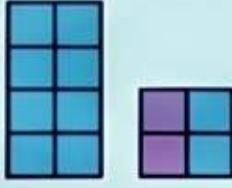
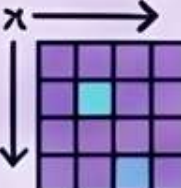
STRATEGIE DI ADDESTRAMENTO PARALLELO

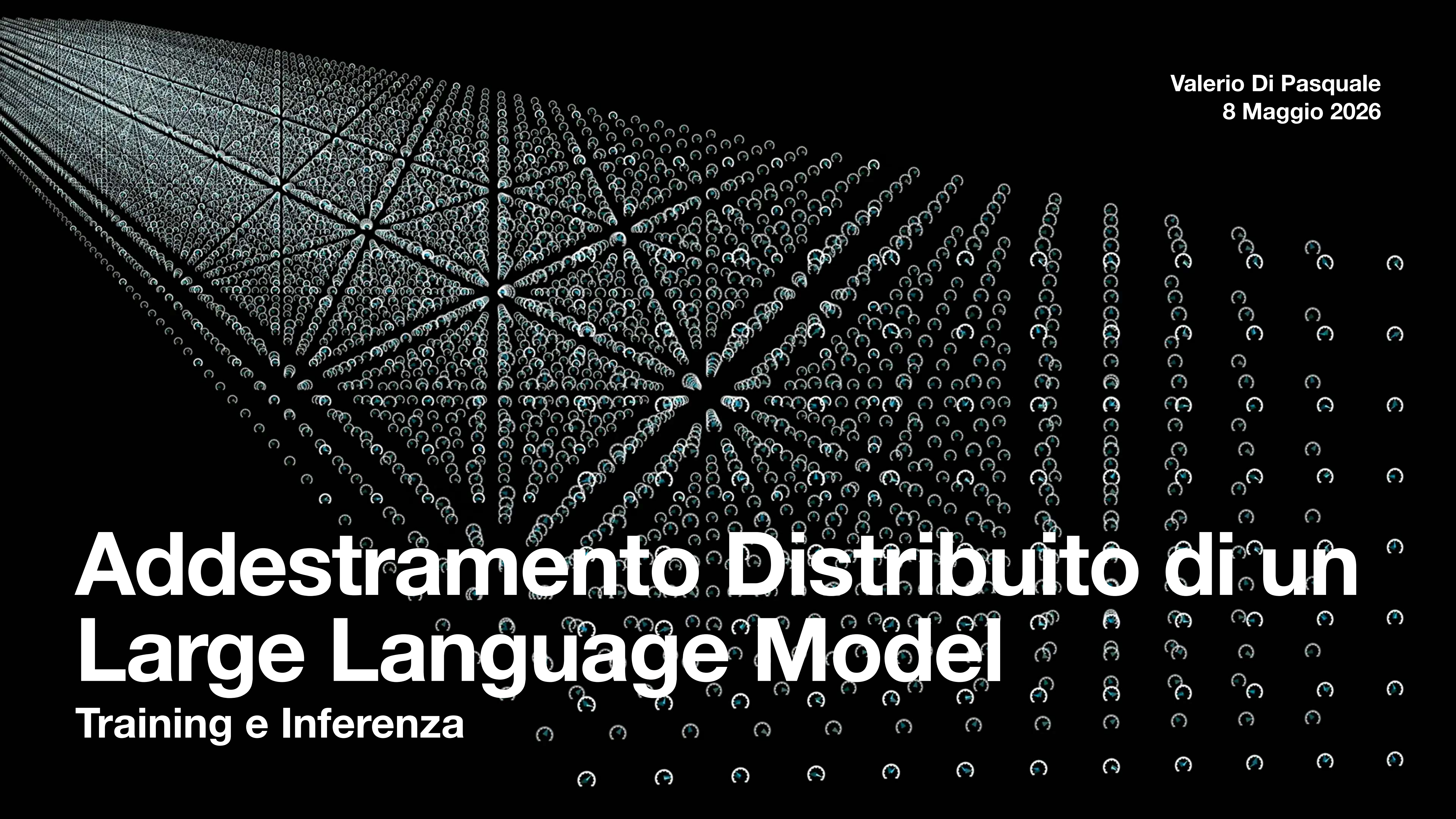


Ottimizzazione della Memoria (ZeRO)

Riduce la ridondanza partizionando stati del modello, gradienti e parametri tra i dispositivi.

SINTESI COMPARATIVA TECNICHE

TECNICA	COSA DIVIDE	QUANDO USARLA
Data Parallelism	 Batch/Dati	Dataset di grandi dimensioni
Model Parallelism	 Layer/Blocchi	Modelli troppo grandi per una sola GPU
Tensor Parallelism	 Operazioni Tensoriali	Modelli con layer estremamente ampi



Valerio Di Pasquale
8 Maggio 2026

Addestramento Distribuito di un Large Language Model

Training e Inferenza

Addestramento Distribuito di un Large Language Model

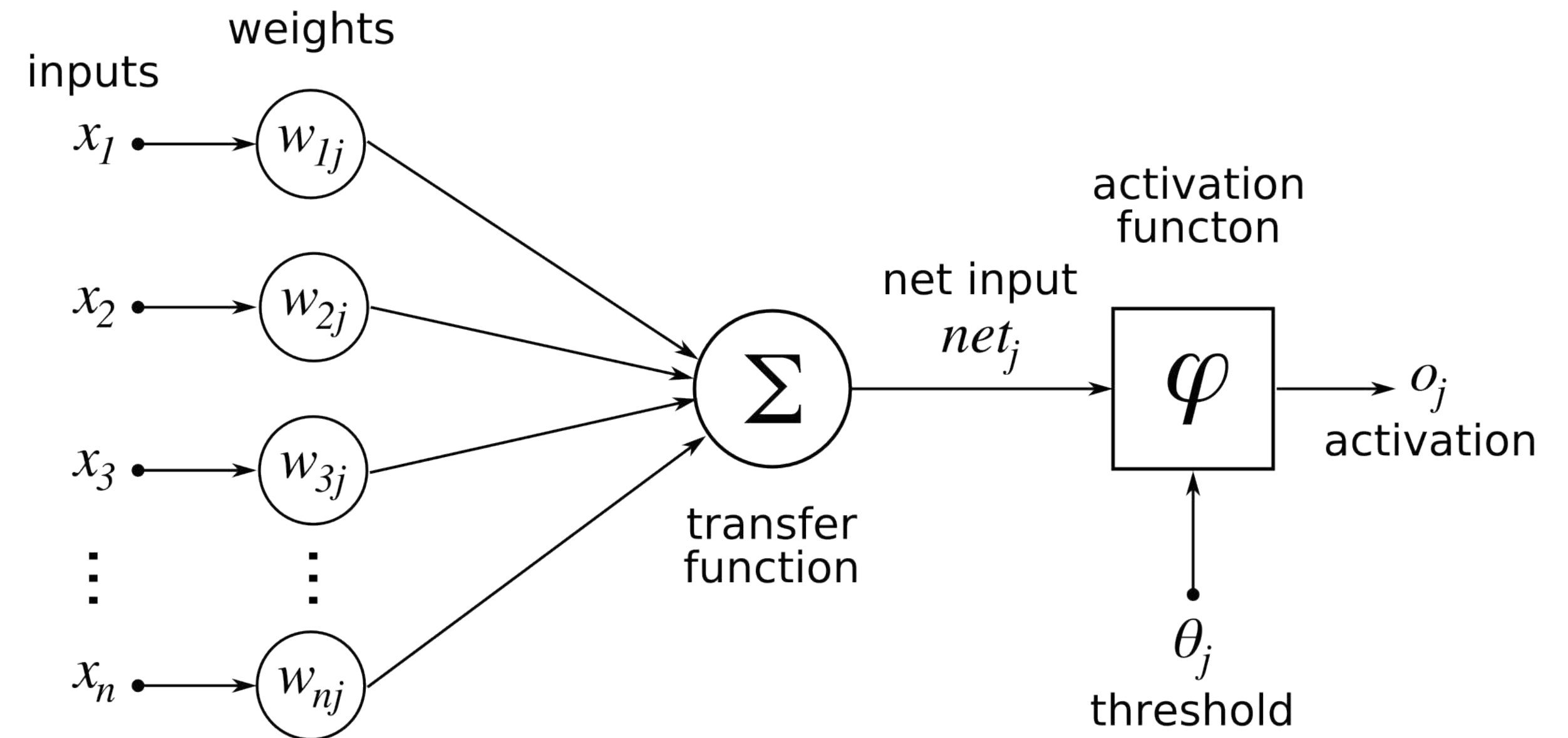
Dai Trasformer ai Large Language Model di Grandi dimensioni

- Richiami alle reti neurali
 - Language Models
 - Reti neurali basate su Transformer
- PyTorch
 - PyTorch Lightning
- Addestramento di un LLM in ambiente distribuito
- Monitorare l'addestramento con Tensorboard

Reti Neurali

Richiamo

- Una rete neurale è un **modello** che **approssima funzioni** complesse attraverso combinazioni di operazioni matematiche, riconoscendo gli **schemi nei dati**.

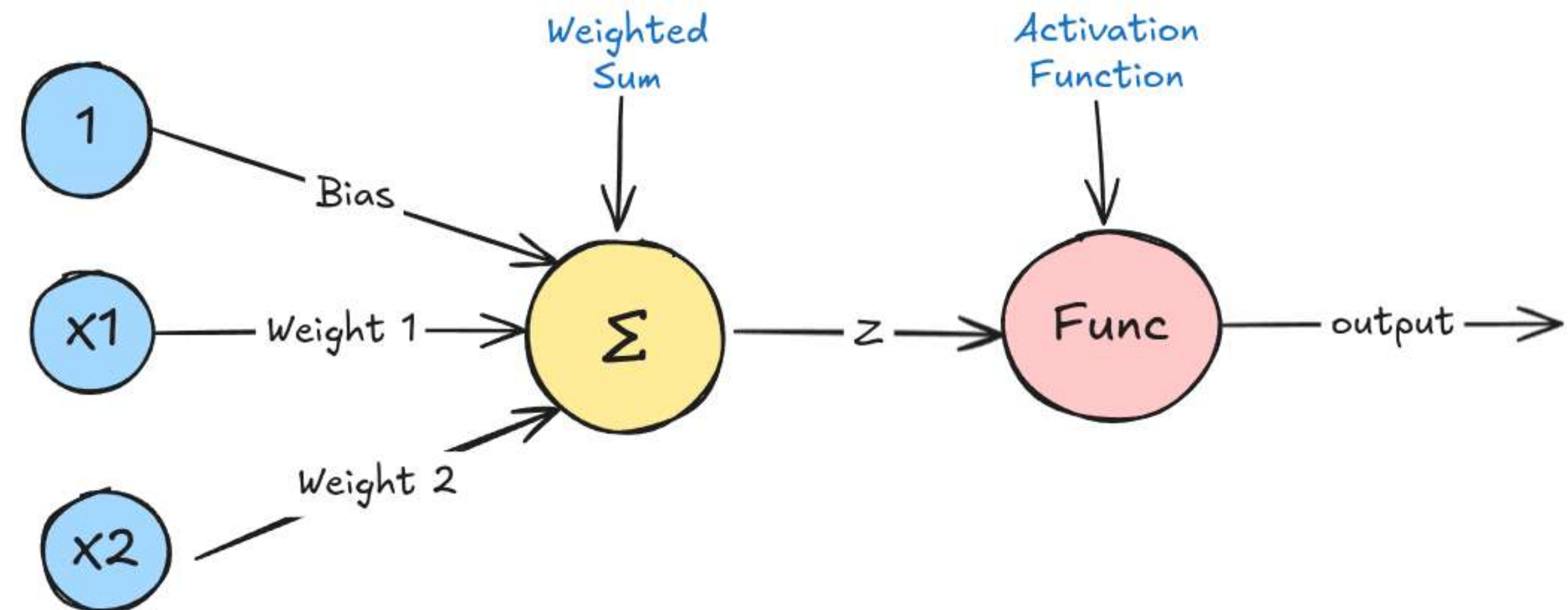


Reti Neurali

Esempio

- I livelli delle reti neurali possono avere funzionalità differenti:
 - Un semplice esempio è il **Percettore**
 - Ideato a partire dal un modello di neurone umano
- Il risultato è funzione di una somma pesata dell'input

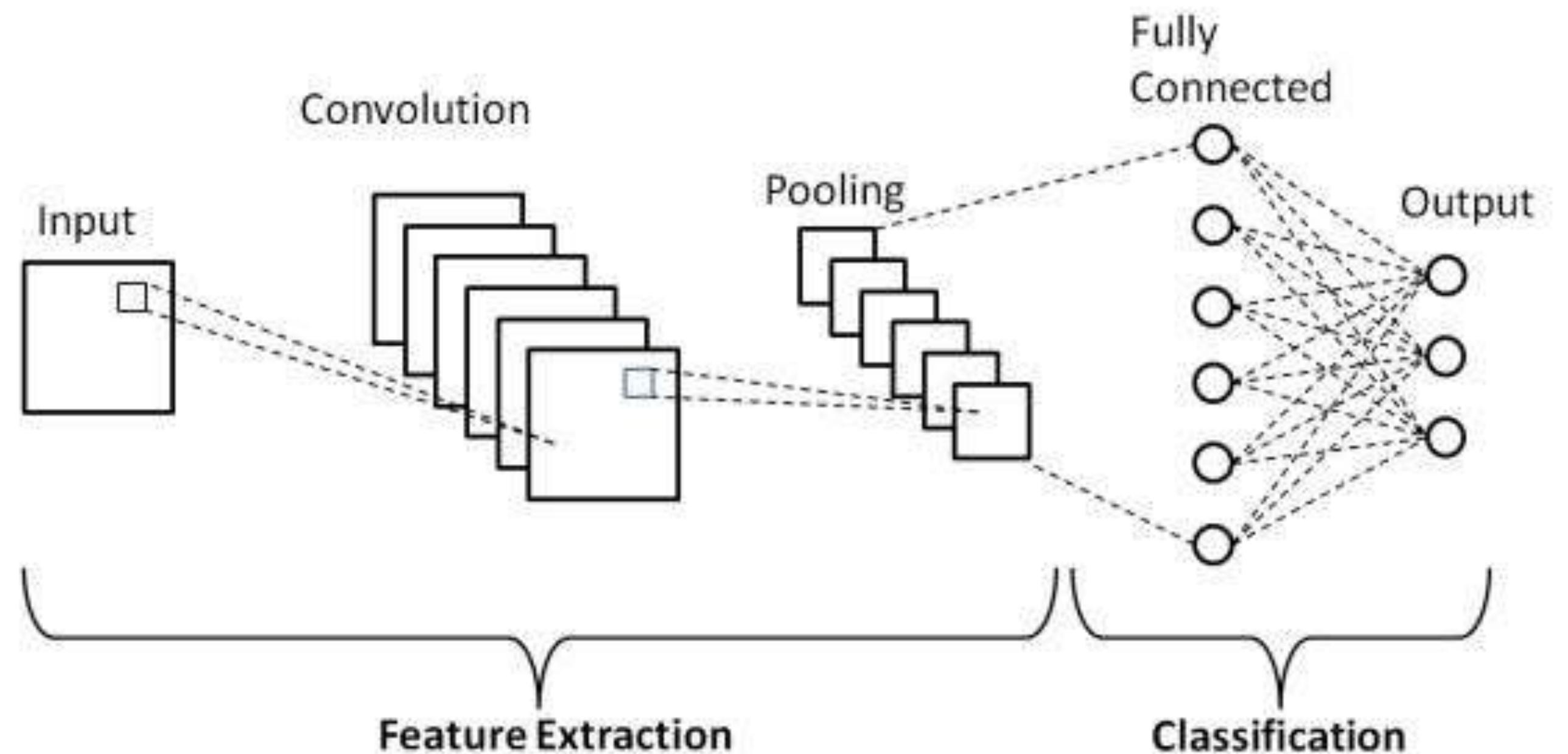
$$\hat{y} = f(Wx + b)$$



Reti Neurali

Esempio

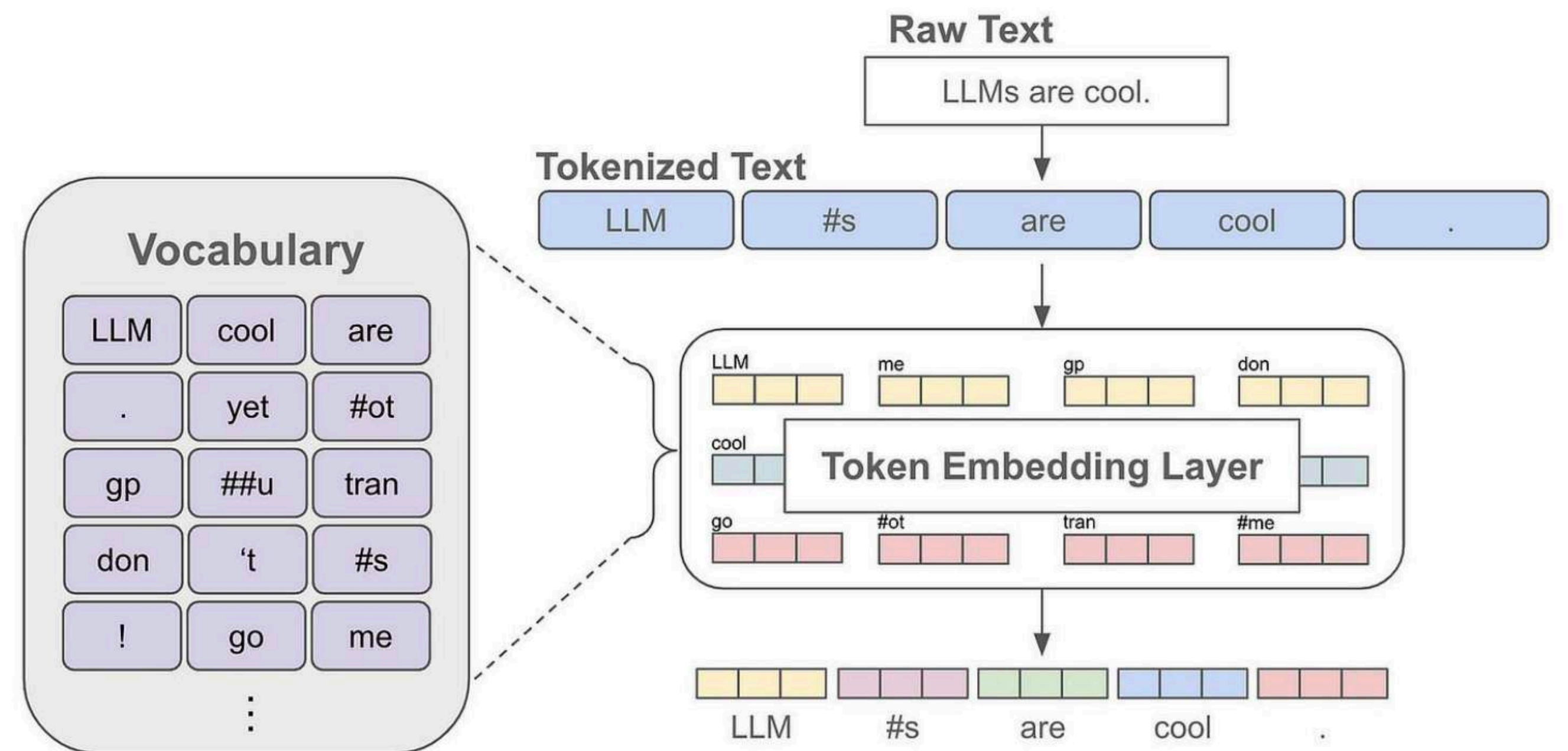
- Un altro esempio di rete neurale sono le **Convolutional Neural Network**
- Comunemente applicate per elaborare le immagini



Language Models

Generazione del testo

- Una rete neurale che genera testo è solitamente addestrata in modo **autoregressivo**:
- L'input della rete è una stringa troncata
- Il modello deve prevedere il prossimo elemento della stringa



x = The quick brown fox jumps over the lazy dog

Language Models

Addestramento autoregressivo

x = The quick brown fox jumps over the lazy y = dog

x = The quick brown fox jumps over the y = lazy

x = The quick brown fox jumps over y = the

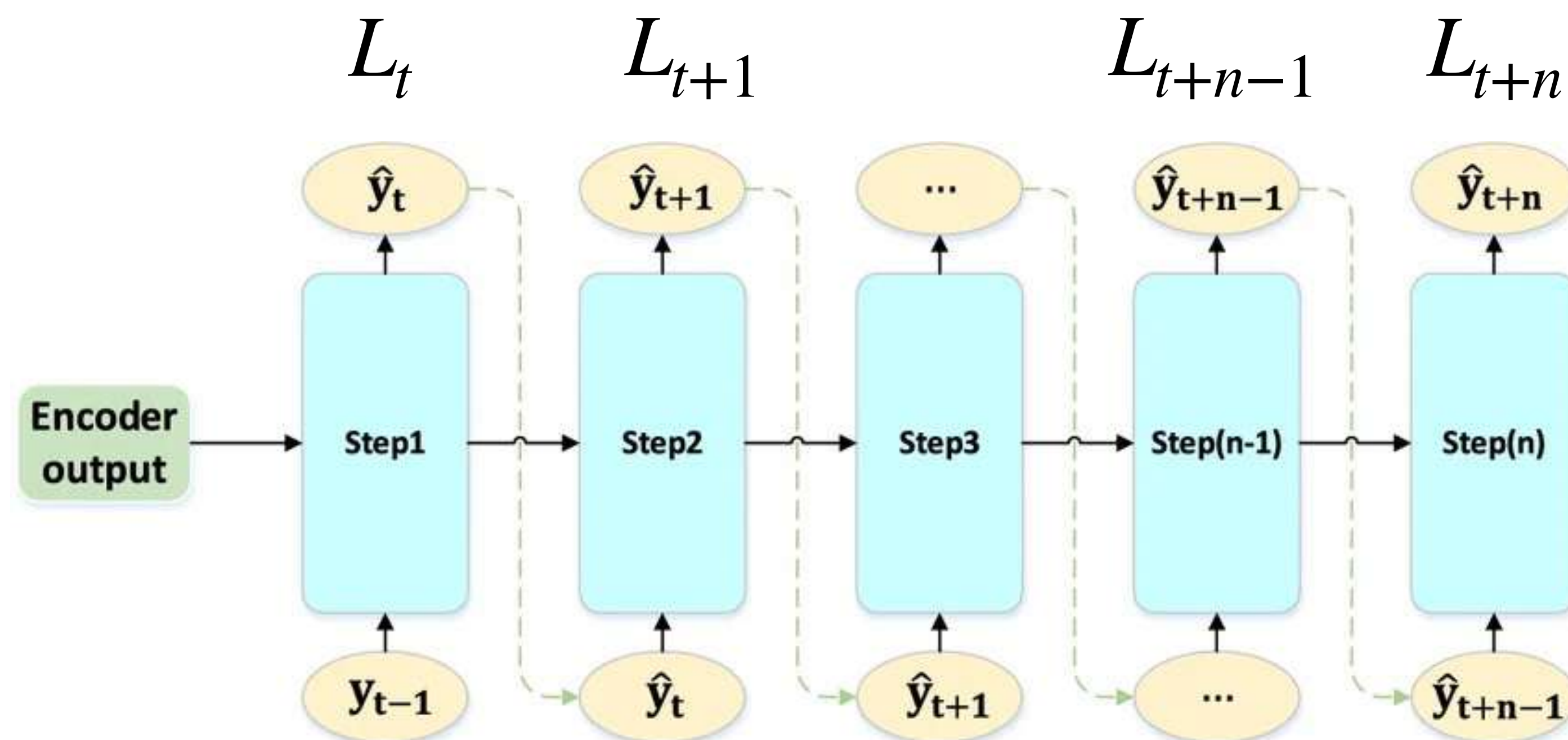
x = The quick brown fox jumps y = over

x = The quick brown fox. y = jumps

Language Models

Generazione del testo

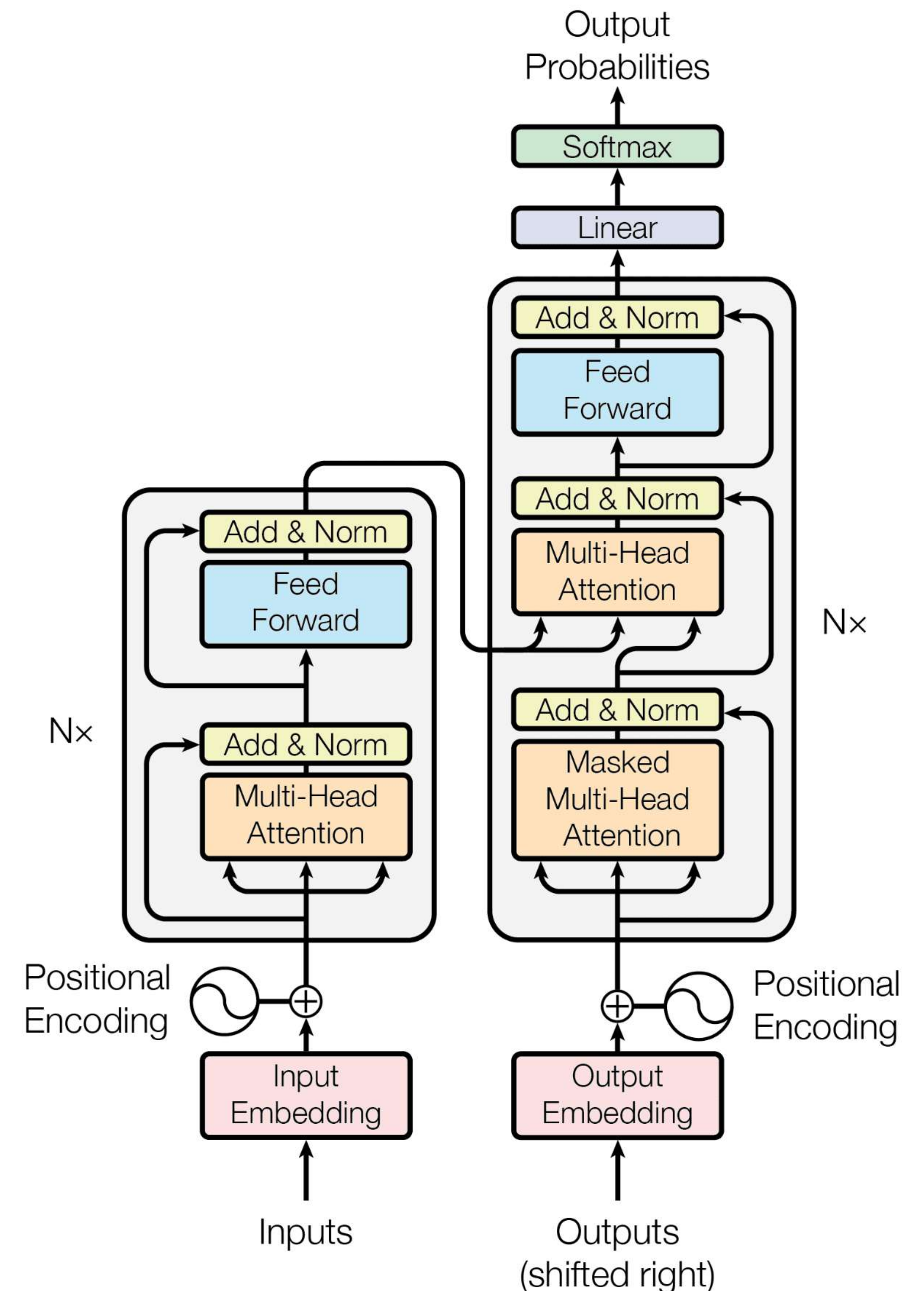
- Nella pratica: il testo viene suddiviso in unità più piccole chiamate **token**
- Il modello riceve in input la stringa fino al token y_{t-1}
- Deve prevedere il token y_t



Transformer

Google 2017

- Le reti neurali comunemente utilizzate per questo task si basano sulla architettura **Transformer**
- La sua idea centrale è il **meccanismo di attention**
- Invece di leggere una frase parola per parola in ordine. La sequenza viene processata contemporaneamente



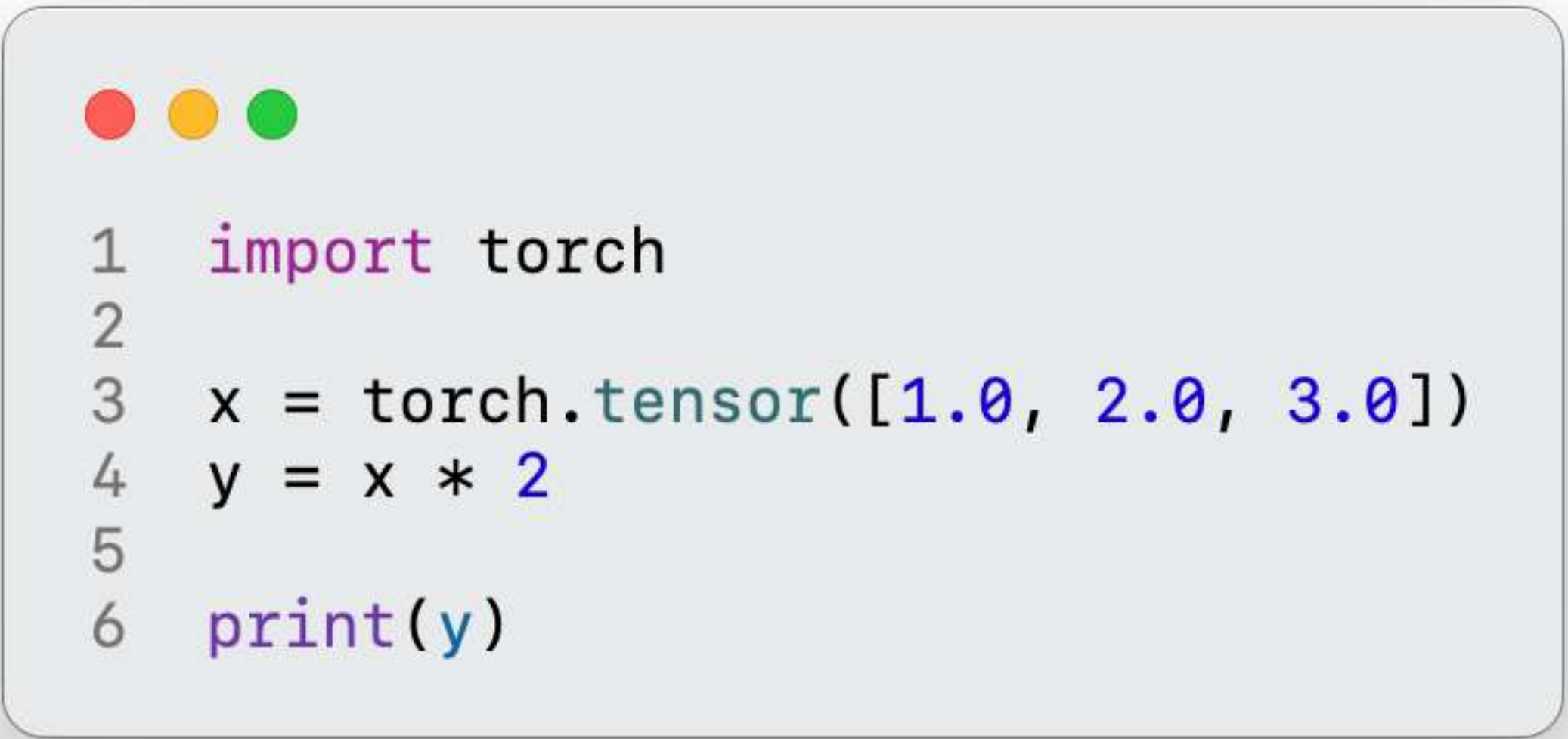
Queste e molte altre tipologie di reti neurali sono già implementate o possono essere implementate facilmente in librerie come **Tensorflow** e **PyTorch**



```
1 > pip install torch|
```

PyTorch

- PyTorch è una libreria open source per il machine learning e il deep learning, usata soprattutto per:
 - **Costruire** reti neurali
 - **Addestrare** reti neurali
 - **Distribuire** reti neurali



```
1 import torch
2
3 x = torch.tensor([1.0, 2.0, 3.0])
4 y = x * 2
5
6 print(y)
```


PyTorch

Costruire reti neurali

- In PyTorch le reti neurali sono definite come classi che estendono `nn.Module`



```
01 # Definizione di una rete neurale
02 class SimpleNet(nn.Module):
03     def __init__(self):
04         super().__init__()
05         self.layer = nn.Linear(1, 1) # 1 input -> 1 output
06
07     def forward(self, x):
08         return self.layer(x)
09
10 model = SimpleNet()
```

PyTorch

Costruire reti neurali

- Definendo il solo **ciclo di addestramento**, la libreria gestirà autonomamente:
 - Gradienti
 - Stati dell'ottimizzazione
 - Aggiornamento dei pesi

```
01 # Funzione di errore e ottimizzatore
02 loss_fn = nn.MSELoss()
03 optimizer = optim.SGD(model.parameters(), lr=0.01)
04 |
05 # Addestramento
06 for epoch in range(1000):
07     prediction = model(x_train)
08     loss = loss_fn(prediction, y_train)
09
10     optimizer.zero_grad()
11     loss.backward()
12     optimizer.step()
13
14 # Test
15 x_test = torch.tensor([[5.0]])
16 y_pred = model(x_test)
```




PyTorch Lightning



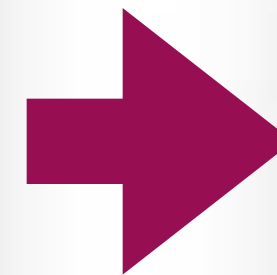
```
1 > pip install lightning
```

PyTorch Lightning

- **PyTorch Lightning** è un *framework* costruito sopra **PyTorch** che serve a organizzare meglio il codice di training dei modelli di deep learning.
 - Aiuta a scrivere codice più pulito e modulare;
 - La complessità viene spostata all'interno del **Trainer**
 - L'oggetto del *framework* che gestisce il ciclo di addestramento



```
1 for epoch in range(num_epochs):  
2     for batch in train_loader:  
3         optimizer.zero_grad()  
4         loss = model(batch)  
5         loss.backward()  
6         optimizer.step()
```



```
1 trainer = L.Trainer(max_epochs=10)  
2 trainer.fit(model, train_loader)
```


PyTorch Lightning

- In **PyTorch**, per fare training su più GPU o più nodi devi
 - Gestire manualmente
 - Inizializzazione dei processi,
 - Sincronizzazione dei gradienti
 - Scelta del backend distribuito
 - Spostamento del modello sui device corretti
 - Salvataggio dei checkpoint solo dal processo principale
 - Logging duplicato

PyTorch Lightning

- Con **Lightning**, invece, tutto ciò può essere controllato tramite i parametri del trainer:
 - Numero di device
 - Numero di processori
 - Backend di comunicazione
 - Strategia di training

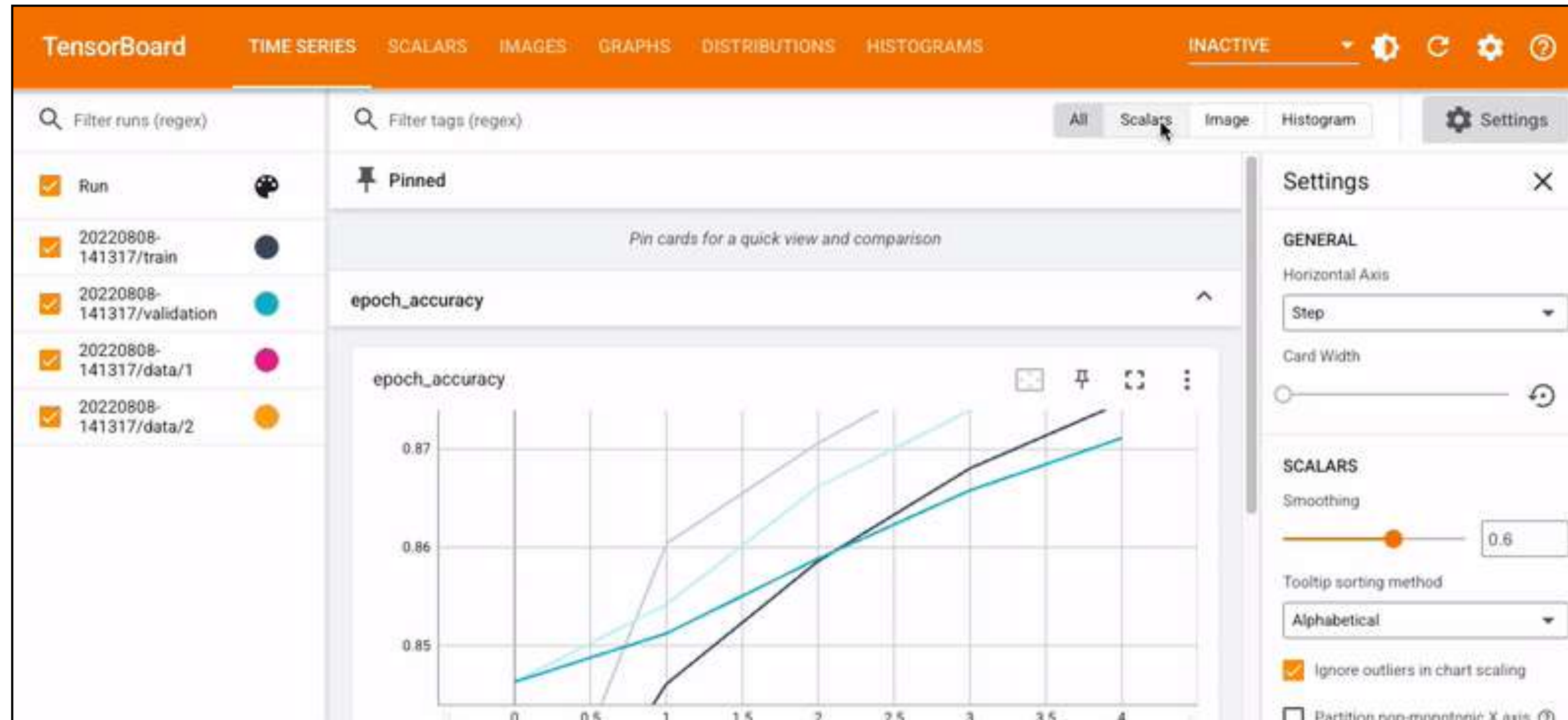
```
01 import lightning as L
02
03 trainer = L.Trainer(
04     accelerator="gpu",
05     devices=4,
06     strategy="ddp",
07     max_epochs=10
08 )
09
10 trainer.fit(model, train_loader, val_loader)
```


Tensorboard

- TensorBoard è uno strumento di visualizzazione usato per **monitorare e analizzare l'addestramento dei modelli**.



```
1 > pip install tensorboard
```



Esercitazione 1

Training distribuito di un LLM

- Il modello che andremo ad addestrare:
 - 4 layer Transformer
- L'addestramento sarà distribuito su 3 nodi in cloud usando la strategia DDP (Distributed Data Parallelism).



```
1 https://github.com/daqh/pcpc-distributed-training
```


Esercitazione 1.1

Modifica al backend

- Modifichiamo il backend per il training del modello in modo da usare **OpenMPI** come backend per la comunicazione.
- Verifichiamo in che modo cambiano i tempi di training.



```
1  trainer = Trainer(  
2      ...,  
3      strategy=DDPStrategy(process_group_backend="mpi")  
4  )
```



```
01  git clone --recursive https://github.com/pytorch/pytorch  
02  cd pytorch  
03  git checkout v2.9.0  
04  git submodule sync  
05  git submodule update --init --recursive  
06  export USE_CUDA=0  
07  export USE_ROCM=0  
08  export USE_MPI=1  
09  export CMAKE_PREFIX_PATH=\"$(python -c 'import sysconfig; print(sysconfig.get_paths()[\"purelib\"])\")\"  
10  python setup.py develop
```

Esercitazione 1.2

Training distribuito di un LLM

- Modificare la taglia del modello aumento il numero di layer a 32
- Modificare i parametri del Trainer addestrando il modello rispetto alla strategia che parallelizzi il modello

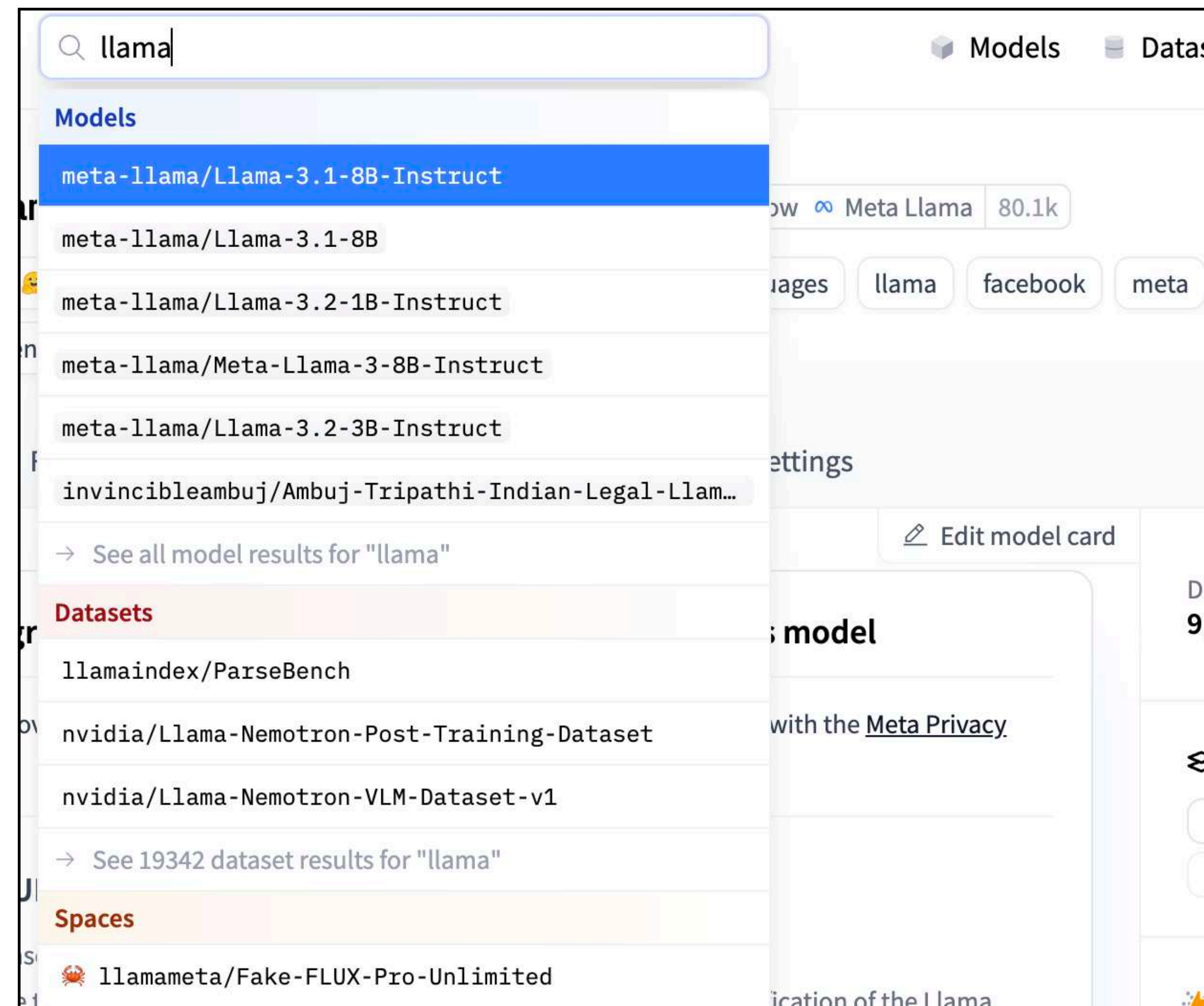
```
01 import lightning as L
02 from lightning.pytorch.strategies import FSDPStrategy
03 from torch.distributed.fsdp.wrap import size_based_auto_wrap_policy
04 from functools import partial
05
06 strategy = FSDPStrategy(
07     auto_wrap_policy=partial(
08         size_based_auto_wrap_policy,
09         min_num_params=1_000_000,
10     ),
11     activation_checkpointing_policy=None,
12 )
13
14 trainer = L.Trainer(
15     accelerator="gpu",
16     devices=4,
17     strategy=strategy,
18     precision="bf16-mixed",
19     max_epochs=10,
20 )
21
22 trainer.fit(model, train_dataloader)
```


Valerio Di Pasquale
14 Maggio 2026

Fine-tuning Distribuito di Modelli per la Generazione di Immagini

Hugging Face 🤗

- Hugging Face è una piattaforma e un ecosistema open source per **lavorare con modelli di machine learning**, soprattutto modelli di NLP, LLM, *computer vision*, audio e modelli multimodali.
- Viene utilizzata da milioni di utenti in tutto il mondo e permette la condivisione di modelli e dataset.



Gradio

- Gradio è una libreria Python per creare rapidamente interfacce web per modelli di machine learning, funzioni Python o demo interattive.
- permette di creare demo pubblicabili su Hugging Face Spaces.

```
01 import gradio as gr
02
03 def saluta(nome):
04     return f"Ciao, {nome}!"
05
06 demo = gr.Interface(
07     fn=saluta,
08     inputs="text",
09     outputs="text"
10 )
11
12 demo.launch()
```

Esercitazione 2

Aumentare le quote e i limiti di Google Cloud

- Per addestrare un modello più grande abbiamo bisogno di **maggiori risorse**
 - IAM e Amministrazione > Quote e Limiti di Sistema > CPUs (all regions)
 - Aumentare la quota totale a 32
- Scaricare il codice dalla repository e costruire il cluster come per l'esercitazione precedente.
 - 3 Numero di nodi
 - 6 CPU per nodo



```
1 https://github.com/daqh/pcpc-distributed-training
```


Esercitazione 2

Fine-tuning Distribuito di un generatore di Canzoni

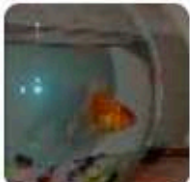
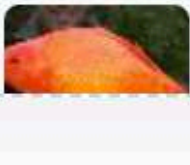
- Il fine-tuning è il processo con cui **partendo da un modello già addestrato e lo si riaddestra su un dataset più specifico per adattarlo a un compito**, uno stile o un dominio particolare.
- Il modello di cui andremo ad eseguire il finetuning è [segmind/tiny-sd](#) (*Stable Diffusion*)
- Lo addestreremo per lavorare meglio su un dataset immagini più ampio
 - Il dataset da utilizzare è [zh-plus/tiny-imagenet](#)
- Sia il modello che il dataset sono pubblici su huggingface
- Scaricare il codice dalla repository e costruire il cluster come per l'esercitazione precedente.
 - 3 Numero di nodi
 - 6 CPU per nodo



```
1 https://github.com/daqh/pcpc-distributed-training
```

Esercitazione 2

Fine-tuning Distribuito di un generatore di Immagini

Dataset Viewer	
Auto-converted to Parquet	
API Embed Duplicate Data Studio	
Split (2) train · 100k rows	
Search is not available for this dataset	
image	label
image · width (px)	class label
64 64	200 classes
	0 n01443537
	0 n01443537
	0 n01443537
	0 n01443537
	0 n01443537



1 <https://github.com/daqh/pcpc-distributed-training>

Esercitazione 2

Fine-tuning Distribuito di un generatore di Immagini

- Modificare sulla macchina locale lo script *train.py* per consentire il training distribuito sul cluster appena creato.
- Una volta effettuate le modifiche, lanciare lo script *./finetune-image.sh*
- Tensorboard -> <http://masternode:80>



```
1  smallaudio/  
2      datamodule.py      # Responsabile della gestione dei dati  
3      model.py           # Logica di addestramento e del modello  
4      requirements.txt   # Dipendenze Python  
5      run-gradio.py      # Server Gradio per l'inferenza  
6      train.py           # Orchestrazione dell'addestramento
```



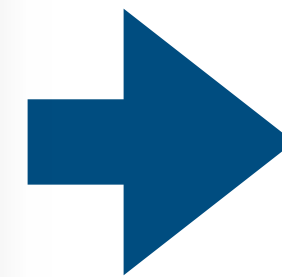
```
1  https://github.com/daqh/pcpc-distributed-training
```

Esercitazione 2

Fine-tuning Distribuito di un generatore di Immagini

- Soluzione

```
01 trainer = L.Trainer(  
02     max_epochs=args.max_epochs,  
03     accelerator="auto",  
04 |  
05     precision="16-mixed" if torch.cuda.is_available() else "32-true",  
06     callbacks=[  
07         checkpoint_callback,  
08     ],  
09     log_every_n_steps=1,  
10 )
```



```
01 trainer = L.Trainer(  
02     max_epochs=args.max_epochs,  
03     accelerator="auto",  
04  
05     devices=6,  
06     num_nodes=3,  
07     strategy="ddp",  
08  
09     precision="16-mixed" if torch.cuda.is_available() else "32-true",  
10     callbacks=[  
11         checkpoint_callback,  
12     ],  
13     log_every_n_steps=1,  
14 )
```


Esercitazione 2.1

Fine-tuning Distribuito di un generatore di Immagini

- Una volta terminato il training eseguire lo script `./run-gradio.sh`
- Il server gradio sarà disponibile al posto di Tensorboard



```
1 https://github.com/daqh/pcpc-distributed-training
```

Esercitazione 2.2

Fine-tuning con un dataset personalizzato

- Usando il formato imagefolder di Hugging Face
- Inserendo una descrizione dei file nel file *metadata.json*

```
1 my_dataset/  
2 | image_001.png  
3 | image_002.png  
4 | image_003.png  
5 | metadata.jsonl
```

```
1 # metadata.json  
2 {"file_name": "image_001.png", "text": "a photo of a red toy car"}  
3 {"file_name": "image_002.png", "text": "a watercolor painting of a castle"}  
4 {"file_name": "image_003.png", "text": "a sketch of a cat wearing sunglasses"}
```


Esercitazione 2

Fine-tuning con un dataset personalizzato

- Come fa questo modello a generare immagini?
 - Utilizza un protocollo estremamente efficace chiamato *Denoising Diffusion*
 - Il modello impara come rimuovere rumore dalle immagini

